



EchoBrains

Empowering Futures, Igniting Careers



AI HOSPITAL READMISSION RISK PREDICTION

List of Contents

Chapter	Page No
CHAPTER 1	4-14
INTRODUCTION	
1.1 Background	4 5
1.2 Problem Statement	6-7
1.3 Proposed Solution	7-8
1.4 Objectives	8-9
1.5 Scope	10-14
CHAPTER 2	15-24
SYSTEM ANALYSIS	
2.1 Existing System	15-16
2.2 Limitations of Existing System	16-17
2.3 Proposed System	18-21
2.4 Advantages	21-24
CHAPTER 3	25-37
REQUIREMENTS	
3.1 Functional Requirements	25-29
3.2 Non-Functional Requirements	30-32
3.3 Hardware Requirements	32-33
3.4 Software Requirements	34-37
CHAPTER 4	38-55
SYSTEM DESIGN	
4.1 System Architecture	38-39
4.2 Module Architecture	40-44
4.3 Database Design	45-46
4.4 ER Diagram	47
4.5 Data Flow Diagram	48-49
4.6 System Flow	50-55
CHAPTER 5	56-77
IMPLEMENTATION	
5.1 Frontend	56-60
5.2 Backend	61-63
5.3 Database	63-65
5.4 Authentication	65-66
5.5 Machine Learning	66-70
5.6 Prediction Service	70-72

5.7 Analytics	72-77
CHAPTER 6	78-93
TESTING	
6.1 Unit Testing	78-81
6.2 API Testing	81-83
6.3 Integration Testing	83-85
6.4 UI Testing	85-89
6.5 Test Cases	89-93
CHAPTER 7	94-100
RESULTS	
7.1 Login	94-95
7.2 Dashboard	95-96
7.3 Patient Management	96-97
7.4 Prediction	97-98
7.5 Prediction History	99
7.6 Analytics Dashboard	100
CHAPTER 8	101-104
ADVANTAGES & LIMITATIONS	
CHAPTER 9	105-107
FUTURE ENHANCEMENTS	
CHAPTER 10	108-109
CONCLUSION	
REFERENCES	110

ABSTRACT

Hospital readmission within 30 days of discharge is an important operational and clinical risk because it may indicate unresolved health needs, complex comorbidities, inadequate follow-up, or other patient and care-process factors. Manual identification of high-risk patients is time-consuming and can be inconsistent when hospitals manage large numbers of admissions. This project presents an integrated AI-Based Hospital Readmission Prediction System that combines relational healthcare data, machine learning, a FastAPI backend, a SQL database, authentication, live prediction, prediction history, and analytics.

The project uses an India Hospital Readmission Dataset covering 2015–2024 with approximately 120,000 admissions and five related tables: admissions, billing, diagnoses, hospitals, and patients. The data is integrated and transformed into a machine-learning dataset. A time-based split is used, with historical records for training and later records for testing, thereby providing a more realistic evaluation setting than a purely random split. The target variable is 30-day readmission. The final primary model is Logistic Regression with balanced class weighting, supported by a preprocessing pipeline consisting of missing-value handling, scaling, and one-hot encoding.

The trained model produces a readmission probability rather than only a binary class. For application-level interpretation, the probability is converted into LOW, MEDIUM, or HIGH risk bands. The live prediction service accepts patient and admission information, validates the request, applies the saved preprocessing pipeline, generates the probability, identifies relevant risk factors, and stores the prediction for later review. A React-based frontend provides login, dashboard, patient management, prediction, history, and analytics functions, while FastAPI exposes secure REST endpoints.

The system demonstrates how machine learning can be embedded into a practical healthcare application rather than being used as an isolated notebook model. The application supports early identification, consistent risk assessment, historical analysis, and decision support. It is intended as an assistive system and not as a replacement for qualified clinical judgment.

Keywords: Hospital Readmission, Machine Learning, Logistic Regression, Healthcare Analytics, FastAPI, React, SQL, Live Prediction, Risk Stratification, Explainable AI.

CHAPTER 1

INTRODUCTION

Healthcare organizations generate large volumes of patient-related data through hospital admissions, diagnoses, laboratory investigations, billing records, treatment histories, insurance information, and previous medical encounters. Proper analysis of this data can help healthcare professionals make better decisions, improve patient outcomes, and optimize hospital resources.

One important challenge faced by hospitals is patient readmission, particularly when a patient is readmitted within 30 days after discharge. Frequent or unexpected readmissions can increase healthcare costs, place additional pressure on hospital resources, and may indicate that a patient requires additional monitoring or follow-up care. Therefore, identifying patients who are at higher risk of readmission before discharge can help healthcare professionals take preventive actions.

The AI-Based Hospital Readmission Prediction System is developed to address this problem using Machine Learning (ML) and data-driven analysis. The system analyzes relevant patient, admission, diagnosis, hospital, and billing information to estimate the probability that a patient will be readmitted within 30 days.

The project uses a relational healthcare dataset containing approximately 120,000 hospital admissions covering the period from 2015 to 2024. The data is organized across multiple tables, including patient information, admissions, diagnoses, hospitals, and billing details. These datasets are integrated and transformed into a machine-learning-ready dataset through data preprocessing and feature engineering.

Several machine-learning algorithms were evaluated during the development of the system, including Random Forest, Gradient Boosting, XGBoost, and Logistic Regression. Logistic Regression was selected as the primary prediction model based on its overall performance and suitability for producing interpretable probability-based predictions. The model achieved a ROC-AUC of 0.7668 on the time-based test dataset.

The system is designed as a complete application rather than only a machine-learning model. It provides a FastAPI backend, database integration, authentication, patient and admission management, live prediction functionality, prediction history, and analytics. A web-based frontend allows authorized users to interact with the system and view prediction results in an understandable format.

The prediction result includes the patient's estimated readmission probability, risk level, department, diagnosis type, relevant risk factors, clinical summary, and recommendations. The system categorizes patients into Low, Medium, and High-risk groups, allowing healthcare staff to identify patients who may require additional attention.

Overall, the project demonstrates how healthcare data, machine learning, backend APIs, databases, authentication, and frontend technologies can be integrated into a practical healthcare decision-support application. The objective is not to replace medical professionals but to provide

an additional data-driven tool that can support early risk identification and help hospitals plan appropriate follow-up and preventive measures.

1.1 Background

The healthcare industry generates a large amount of structured and unstructured data through patient registration, hospital admissions, diagnoses, laboratory investigations, treatment records, billing transactions, insurance information, and previous medical encounters. The availability of such data provides an opportunity to apply Artificial Intelligence (AI) and Machine Learning (ML) techniques to support healthcare decision-making.

One of the important challenges faced by hospitals is **patient readmission**. Hospital readmission refers to a situation in which a patient returns to the hospital for further treatment after being discharged. In particular, **30-day readmission** is an important healthcare quality indicator because an early readmission may indicate that a patient requires additional monitoring, follow-up, or post-discharge care.

Unplanned readmissions can increase healthcare expenditure and place additional pressure on hospital infrastructure, medical staff, beds, and other resources. Identifying patients who have a higher probability of readmission before discharge can therefore help healthcare organizations take preventive measures.

Traditional approaches to identifying readmission risk often depend on clinical experience, manually reviewing patient records, and applying fixed rules. Although these approaches are useful, they can become difficult to maintain when the number of patients and medical records increases. Machine Learning provides an alternative approach by learning patterns from historical patient data and using those patterns to estimate the likelihood of future outcomes.

The proposed **AI-Based Hospital Readmission Prediction System** applies Machine Learning to historical hospital data to predict whether a patient is likely to be readmitted within 30 days. The project integrates information from multiple relational datasets, including patient details, admission records, diagnoses, hospital information, and billing information.

The system follows a complete data science and application development pipeline consisting of data collection, data integration, preprocessing, feature engineering, model training, model evaluation, prediction, API development, database integration, authentication, and frontend visualization.

The dataset used in the project contains approximately **120,000 hospital admissions covering the period from 2015 to 2024**. The data contains information such as patient demographics, previous admissions, comorbidities, diagnosis information, hospital details, billing-related information, and the 30-day readmission outcome.

Multiple machine-learning algorithms were evaluated, including **Random Forest, Gradient Boosting, XGBoost, and Logistic Regression**. Logistic Regression was selected as the primary model because it provided the highest ROC-AUC among the evaluated models and produces probability-based predictions that can be interpreted easily.

The trained model is integrated into a live prediction service through a **FastAPI backend**. Authorized users can enter patient and admission information through the web application and obtain a real-time readmission-risk prediction. The system presents the prediction as a probability and categorizes the result into Low, Medium, or High risk.

Thus, the project demonstrates the practical application of Machine Learning in healthcare by combining predictive analytics with a complete web-based information system.

1.2 Problem Statement

Hospital readmission is a significant challenge for healthcare organizations because patients who return to the hospital shortly after discharge may require additional medical care and hospital resources. Identifying such patients in advance can help healthcare professionals provide appropriate follow-up and preventive care.

In conventional hospital environments, identifying readmission risk may depend heavily on manual analysis of patient records and the experience of healthcare professionals. When hospitals manage a large number of patients, manually examining multiple factors such as previous admissions, diagnosis, comorbidities, demographic information, and hospital-related information can be time-consuming and difficult.

Another challenge is that patient information is often distributed across different datasets or database tables. Patient details, admission records, diagnoses, hospital information, and billing information may need to be combined before meaningful analysis can be performed.

Therefore, there is a need for an automated system that can:

- Integrate information from multiple healthcare data sources.
- Preprocess and transform raw healthcare data.
- Identify important factors associated with readmission.
- Use historical data to train a machine-learning model.
- Estimate the probability of 30-day readmission.
- Categorize patients according to their predicted risk.
- Provide predictions through a user-friendly application.
- Maintain prediction history for future reference.
- Provide analytics to support hospital-level decision-making.

The **AI-Based Hospital Readmission Prediction System** is developed to address these requirements by combining Machine Learning, FastAPI, database technologies, authentication, and a web-based frontend into a single application.

1.3 Proposed Solution

The proposed system is an AI-powered healthcare decision-support application that predicts the probability of a patient being readmitted within 30 days after hospital discharge.

The system first combines information from multiple healthcare tables. The integrated data is cleaned and transformed into a suitable format for machine-learning processing. Missing values are handled through appropriate preprocessing techniques, while categorical features are encoded and numerical features are scaled.

A time-based approach is used for model evaluation. Historical admissions are used for model training, while later-period admissions are reserved for testing. This provides a more realistic evaluation because the model is tested on data from a later period rather than randomly mixing records from different time periods.

Different machine-learning algorithms are trained and evaluated using metrics such as:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC
- Confusion Matrix

The final Logistic Regression model generates a probability representing the estimated likelihood of 30-day readmission.

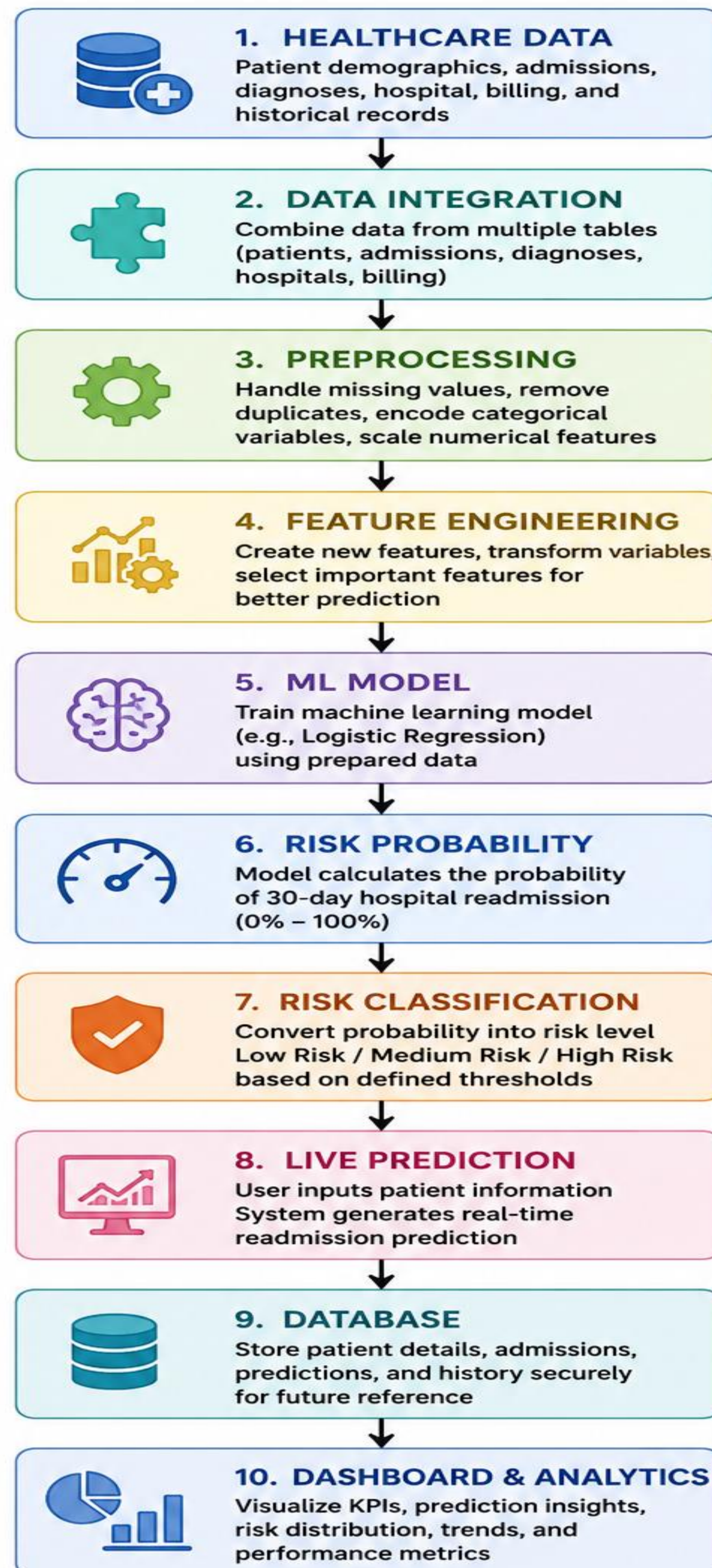
The application then converts the probability into a risk category:

Risk Level	Probability Range
Low Risk	Less than 35%
Medium Risk	35%-54%
High Risk	55% or above

The prediction service is exposed through FastAPI APIs. The frontend communicates with the backend through these APIs, allowing users to submit patient information and receive live predictions.

The system also stores relevant patient, admission, and prediction information in the database. Users can view previous predictions and access analytics through the dashboard.

The overall solution can be represented as:



The system is intended to support healthcare professionals rather than replace clinical judgment. The prediction should be considered a decision-support output that can assist professionals in identifying patients who may require additional attention.

1.4 Objectives

The major objective of the project is to develop an intelligent hospital readmission prediction system capable of estimating the probability of 30-day patient readmission using historical healthcare data.

1.4.1 Primary Objective

To design and develop a Machine Learning-based web application that predicts the risk of hospital readmission within 30 days and provides the prediction through a live, user-friendly interface.

1.4.2 Specific Objectives

The specific objectives of the project are:

1. **To collect and integrate healthcare data**
Combine patient, admission, diagnosis, hospital, and billing information into a unified dataset suitable for analysis.
2. **To preprocess healthcare data**
Handle missing values, transform categorical variables, scale numerical features, and prepare the dataset for machine-learning algorithms.
3. **To perform feature engineering**
Identify and construct useful predictive features from available healthcare information.
4. **To analyze the readmission problem**
Study the distribution of readmitted and non-readmitted patients and identify relevant patterns within the dataset.
5. **To develop machine-learning models**
Train and compare different algorithms, including Random Forest, Gradient Boosting, XGBoost, and Logistic Regression.
6. **To evaluate model performance**
Compare the models using Accuracy, Precision, Recall, F1-score, ROC-AUC, and Confusion Matrix.
7. **To select an appropriate prediction model**
Select Logistic Regression as the primary model based on its evaluation performance and suitability for probability-based risk prediction.
8. **To implement live prediction**
Develop an API-based prediction service that can process new patient information and generate a prediction in real time.
9. **To classify readmission risk**
Convert predicted probabilities into Low, Medium, and High-risk categories to make results easier to understand.

10. To provide risk-related information

Display relevant risk factors, diagnosis information, clinical summary, and recommendations along with the prediction.

11. To implement secure authentication

Provide user authentication and role-based access to protect application functionality and healthcare-related information.

12. To maintain prediction history

Store predictions so that authorized users can review previous prediction results.

13. To develop an analytics dashboard

Provide statistical summaries and visual analytics to help users understand readmission patterns and prediction outcomes.

14. To develop a scalable application architecture

Integrate frontend, backend, database, and machine-learning components into a structured application.

1.5 Scope

The scope of the project covers the development of an end-to-end AI-based hospital readmission prediction application. The system focuses primarily on predicting the likelihood of **30-day hospital readmission** using available historical healthcare information.

1.5.1 Data Scope

The system works with healthcare data containing information related to:

- Patient demographics
- Patient history
- Previous admissions
- Comorbidities
- Hospital admissions
- Diagnosis information
- ICD-10-related information
- Hospital information
- Billing information
- Readmission outcomes

The project integrates information from the five major relational datasets:

1. patients.csv
2. admissions.csv
3. diagnoses.csv
4. hospitals.csv
5. billing.csv

1.5.2 Machine Learning Scope

The Machine Learning component covers:

- Data preprocessing
- Missing-value handling
- Categorical feature encoding
- Numerical feature scaling
- Feature preparation
- Model training
- Model comparison
- Model evaluation
- Probability prediction
- Risk classification
- Threshold analysis

The evaluated algorithms include:

- Logistic Regression
- Random Forest
- Gradient Boosting
- XGBoost

The final system uses Logistic Regression as the primary prediction model.

1.5.3 Application Scope

The web application provides functionality for:

- User login and authentication
- Patient management
- Admission management

- Live readmission prediction
- Prediction history
- Risk-factor display
- Dashboard analytics
- Model-based risk classification

1.5.4 Backend Scope

The backend is responsible for:

- API development
- Authentication
- Request validation
- Database communication
- Machine-learning model loading
- Live prediction
- Prediction storage
- Business logic
- Analytics services

The backend is implemented using **Python and FastAPI**.

1.5.5 Database Scope

The database maintains application information related to:

- Users
- Departments
- Patients
- Admissions
- Predictions

The database enables the application to maintain relationships between patients, admissions, and prediction results.

1.5.6 Frontend Scope

The frontend provides an interactive interface through which authorized users can:

- Log into the application.

- View dashboard information.
- Register and manage patient information.
- View admission details.
- Enter prediction-related information.
- Generate live predictions.
- View risk levels and probabilities.
- Review prediction history.
- Access analytics.

1.5.7 Risk Prediction Scope

The system provides three primary risk categories:

- **Low Risk:** predicted probability below 35%.
- **Medium Risk:** predicted probability from 35% to below 55%.
- **High Risk:** predicted probability of 55% or higher.

These categories are intended to make model output easier for users to interpret.

1.5.8 Limitations of Scope

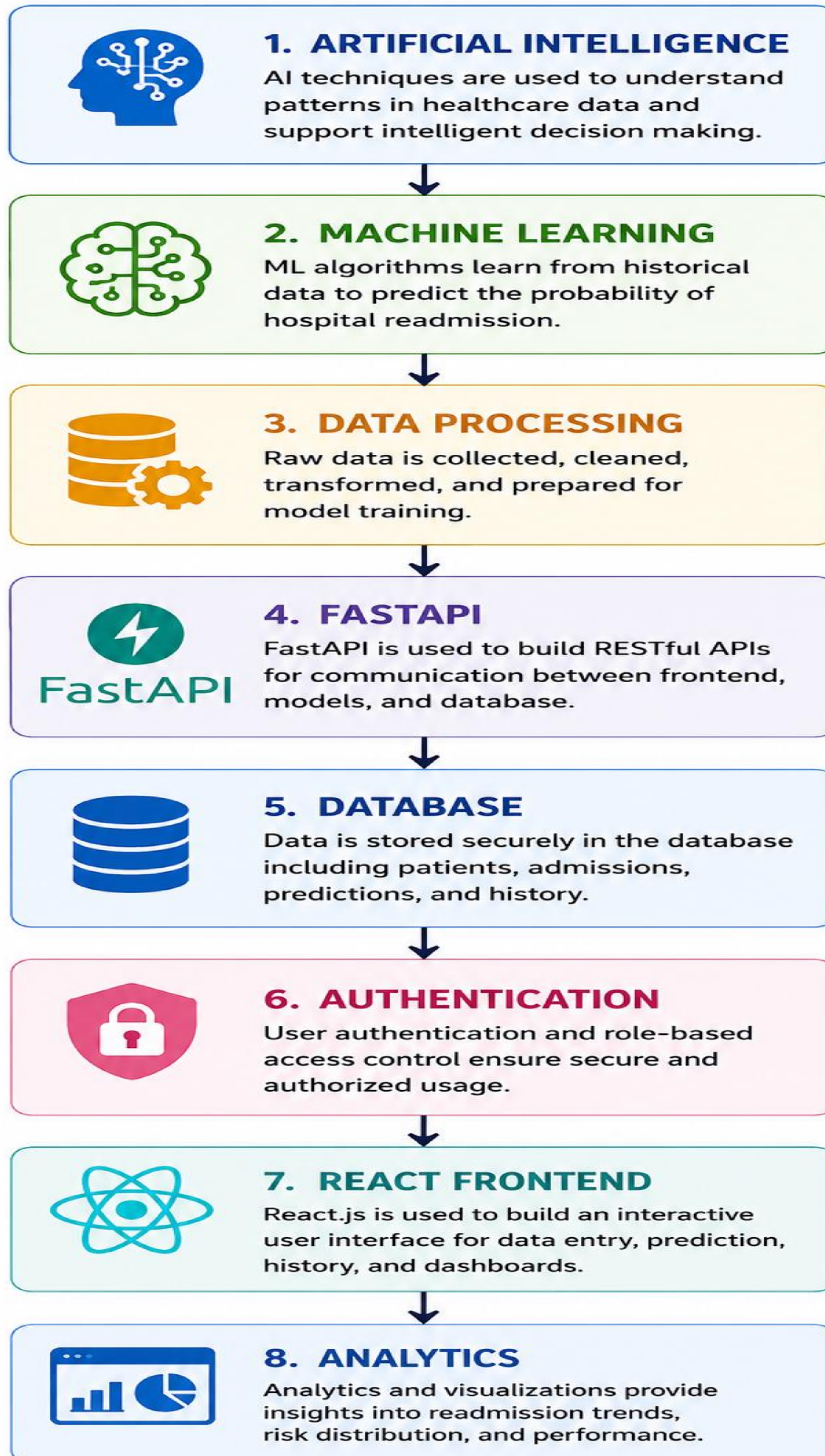
The system is designed as a predictive decision-support application and does not replace doctors or other qualified healthcare professionals. It does not independently diagnose diseases, prescribe medication, or make final clinical decisions.

The prediction quality depends on the quality, completeness, representativeness, and relevance of the historical data used to train the model. Therefore, predictions should be interpreted together with professional clinical assessment.

1.5.9 Overall Scope

The overall scope of the project is to demonstrate how AI and Machine Learning can be integrated with modern software technologies to build a practical healthcare application.

Healthcare Technology Stack Flowchart:



CHAPTER 2

System Analysis

System Analysis is an important phase in the development of the **AI-Based Hospital Readmission Prediction System**. It involves studying the existing healthcare system, identifying its limitations, understanding user and system requirements, and determining how the proposed system can address the identified problems.

The analysis focuses on the challenges involved in managing patient information, hospital admissions, diagnosis records, and historical healthcare data. It also examines the need for an automated Machine Learning-based approach for predicting the probability of patient readmission within 30 days.

In the proposed system, healthcare data from multiple sources is integrated and processed before being provided to the Machine Learning model. The system analyzes relevant patient and admission-related information and generates a probability-based readmission prediction. The prediction is then classified into **Low, Medium, or High risk** and presented through a user-friendly web interface.

System analysis also considers the complete application environment, including the **React frontend, FastAPI backend, database, authentication, Machine Learning model, live prediction service, and analytics dashboard**. This ensures that the project is not limited to model development but functions as an integrated healthcare application.

The main purpose of this chapter is to analyze the existing approach, identify its limitations, describe the proposed solution, and explain the advantages of implementing an AI-based prediction system. This analysis provides the foundation for designing and implementing an efficient, secure, scalable, and user-friendly hospital readmission prediction system.

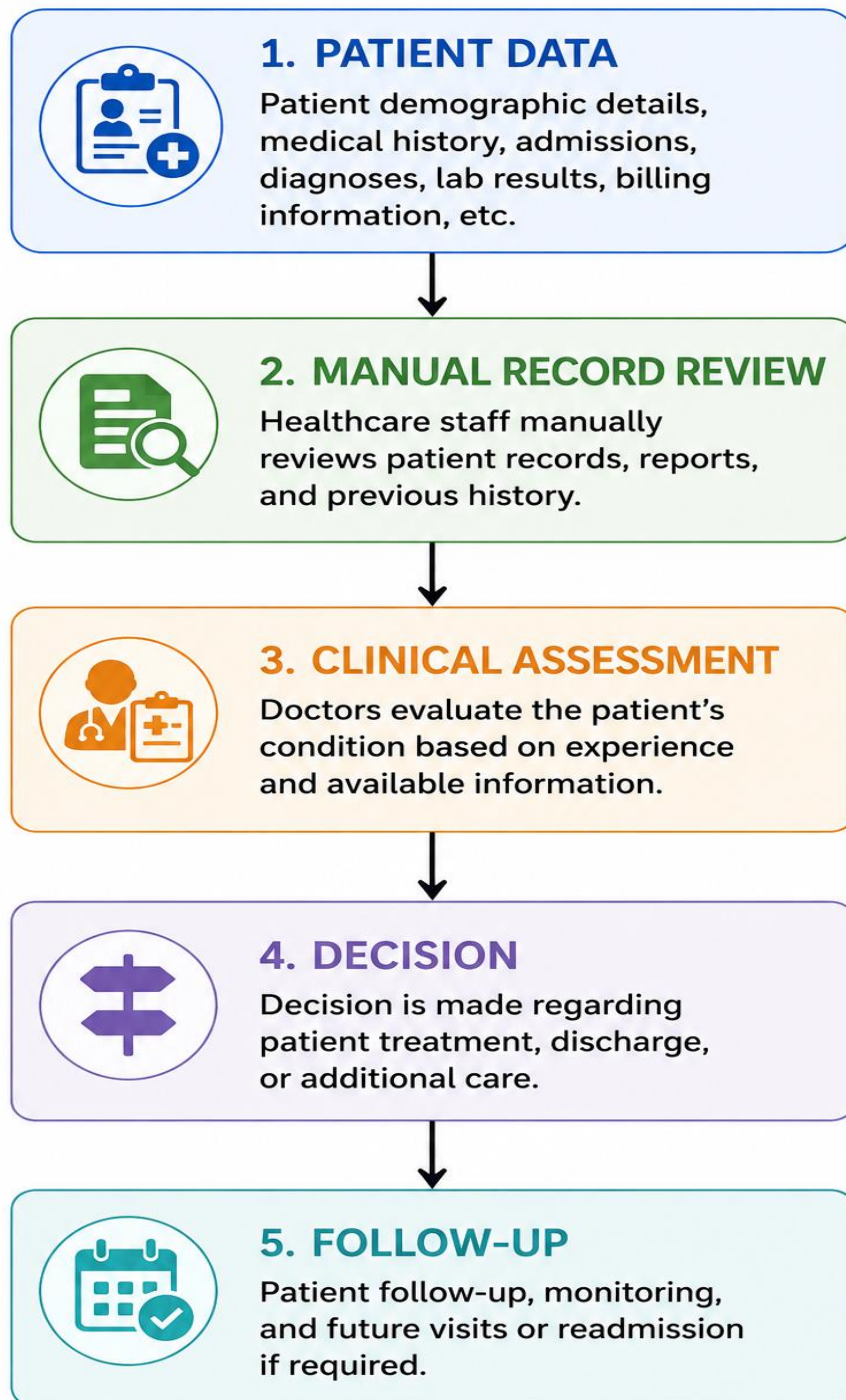
2.1 Existing System

In many conventional healthcare environments, hospital readmission risk is assessed using manual review of patient records, clinical experience, predefined rules, and basic statistical reports. Healthcare professionals examine information such as the patient's previous admissions, diagnosis, age, comorbidities, treatment history, and other clinical factors to determine whether additional follow-up may be required.

In a traditional hospital information system, patient information is generally stored in different modules or records. Admission information, diagnosis details, billing information, and patient history may exist separately. Healthcare staff may need to review multiple records before making an assessment.

Some existing systems provide basic reporting and patient management functionality, but they may not provide an integrated Machine Learning-based prediction mechanism.

The conventional process can be represented as:



Although healthcare professionals remain essential for final decision-making, manually analyzing a large number of patient records can be time-consuming. The increasing volume of healthcare data creates a need for automated analytical and predictive solutions.

Major characteristics of the existing system

- Patient information is maintained through conventional hospital records.
 - Patient history is reviewed manually.
 - Different healthcare data sources may be maintained separately.
 - Readmission risk may be assessed using clinical judgment or predefined criteria.
 - Basic reports can be generated from stored data.
 - There is limited automated prediction of future readmission.
 - Large-scale historical data may not be fully utilized for predictive analysis.
 - Prediction results may not be available through a real-time web interface.
-

2.2 Limitations of Existing System

The conventional approach has several limitations when dealing with a large number of patients and complex healthcare datasets.

2.2.1 Manual Risk Assessment

Healthcare professionals may need to manually review several patient attributes before determining whether a patient could be at risk of readmission. This process can consume considerable time, particularly when patient volume is high.

2.2.2 Difficulty in Handling Large Datasets

Hospitals can generate large quantities of patient and admission data. Manually analyzing thousands of records makes it difficult to identify patterns that may not be immediately visible.

2.2.3 Data Scattered Across Multiple Sources

Patient information, admission records, diagnosis details, hospital information, and billing information may be stored in separate tables or systems. Combining these sources manually can be difficult and error-prone.

2.2.4 Limited Predictive Capability

Traditional hospital information systems are primarily designed for storing and retrieving information. They may not provide an ML-based probability of future readmission.

2.2.5 Lack of Real-Time Prediction

In a conventional system, there may not be a mechanism through which authorized users can enter current patient information and immediately obtain an estimated readmission risk.

2.2.6 Limited Risk Categorization

A basic system may provide patient information without converting analytical results into understandable categories such as **Low Risk, Medium Risk, and High Risk**.

2.2.7 Difficulty Identifying Hidden Patterns

Human analysis may not consistently identify complex relationships among multiple variables. Machine Learning can analyze historical data and identify statistical patterns that can be useful for prediction.

2.2.8 Limited Historical Prediction Analysis

Without a dedicated prediction-history module, it can be difficult to review previous predictions and analyze how risk predictions are distributed over time.

2.2.9 Limited Analytics

Traditional systems may provide basic reports but may not provide an integrated analytics dashboard containing prediction trends, risk distributions, and performance-related information.

Summary of Existing System Limitations

LIMITATION	DESCRIPTION
Manual assessment	Requires considerable human effort
Data fragmentation	Information may exist in multiple sources
Limited prediction	No integrated ML-based readmission prediction
No live prediction	Real-time risk estimation may not be available
Large-data challenge	Difficult to manually analyze large datasets
Limited risk classification	Risk may not be presented in clear categories
Limited analytics	Lack of prediction-oriented dashboards
Limited automation	Several activities require manual intervention

2.3 Proposed System

The proposed **AI-Based Hospital Readmission Prediction System** is an integrated healthcare decision-support application that uses Artificial Intelligence and Machine Learning to estimate the probability of a patient's readmission within 30 days.

The system combines healthcare data from multiple relational sources, including:

- Patient information

- Hospital admission information
- Diagnosis information
- Hospital information
- Billing information

The integrated data is processed through a structured Machine Learning pipeline. Data preprocessing handles missing values and prepares numerical and categorical variables for model training. Feature engineering is then performed to prepare meaningful input variables for prediction.

Several Machine Learning algorithms are evaluated during development:

1. Logistic Regression
2. Random Forest
3. Gradient Boosting
4. XGBoost

The models are evaluated using metrics such as Accuracy, Precision, Recall, F1-score, ROC-AUC, and Confusion Matrix. **Logistic Regression** is selected as the primary prediction model based on the evaluation performed for this project.

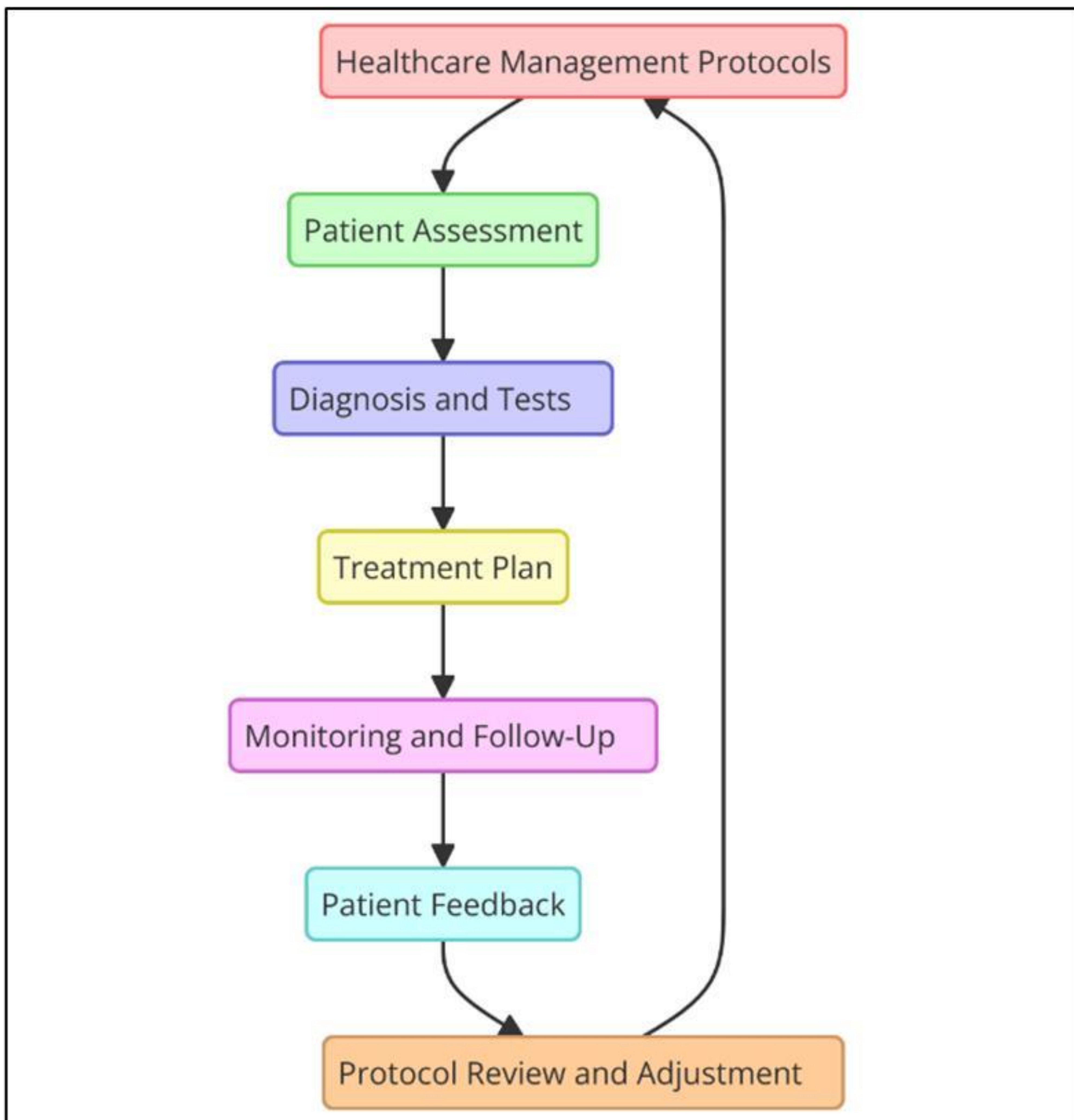
The final model generates a probability representing the estimated likelihood of 30-day readmission. The probability is converted into a risk category to make the output easier to interpret.

Risk Level	Probability Range
Low Risk	Less than 35%
Medium Risk	35%-54%
High Risk	55% or above

The proposed system also provides a **live prediction facility**. Authorized users can enter patient and admission information through the frontend, and the information is sent to the FastAPI backend. The prediction service processes the input using the trained model and returns the predicted probability and risk category.

The prediction information can then be stored in the database and made available through the prediction-history section.

Proposed system workflow



2.3.1 Major Components of the Proposed System

1. Data Integration Module

This module combines information from the different healthcare datasets into a unified dataset. It establishes relationships between patient, admission, diagnosis, hospital, and billing information.

2. Data Preprocessing Module

This module prepares raw healthcare data for Machine Learning by handling missing values, transforming categorical variables, and scaling numerical variables.

3. Machine Learning Module

The ML module trains and evaluates multiple algorithms and selects an appropriate model for readmission prediction.

The final Logistic Regression model achieved a **ROC-AUC of 0.7668** on the project's time-based test dataset.

4. Prediction Service

The prediction service loads the trained model and preprocessing artifacts and accepts new patient information. It generates a real-time probability of 30-day readmission.

5. Risk Classification Module

The predicted probability is converted into Low, Medium, or High-risk categories using the defined thresholds.

6. Patient Management Module

Authorized users can manage patient information and associate patients with their hospital admissions.

7. Admission Management Module

The system maintains admission-related information and associates admissions with corresponding patients and predictions.

8. Authentication Module

The system provides user authentication and role-based access so that application functionality can be accessed by authorized users.

9. Prediction History Module

Previous predictions are stored and can be reviewed by authorized users. This allows users to track prediction results associated with admissions.

10. Analytics Module

The analytics dashboard provides visual summaries and insights related to prediction results, risk distribution, trends, and system performance.

2.4 Advantages of the Proposed System

The proposed system provides several advantages over a conventional manual approach.

2.4.1 Automated Risk Prediction

The system automatically calculates the probability of 30-day readmission using a trained Machine Learning model, reducing dependence on manual risk calculations.

2.4.2 Real-Time Prediction

The live prediction feature allows users to enter patient information and obtain a prediction without waiting for offline model execution.

2.4.3 Integration of Multiple Data Sources

Patient, admission, diagnosis, hospital, and billing information can be integrated into a single analytical workflow.

2.4.4 Data-Driven Decision Support

The system provides a quantitative risk probability that can support healthcare professionals during patient assessment.

2.4.5 Easy-to-Understand Risk Levels

Instead of displaying only a numerical probability, the system classifies the patient into:

Low → Medium → High

This makes the prediction easier to interpret.

2.4.6 Prediction History

The system stores previous prediction results, allowing authorized users to review historical predictions.

2.4.7 Analytics Dashboard

The dashboard provides visual insights into risk distribution, prediction trends, and other system-level information.

2.4.8 Secure Access

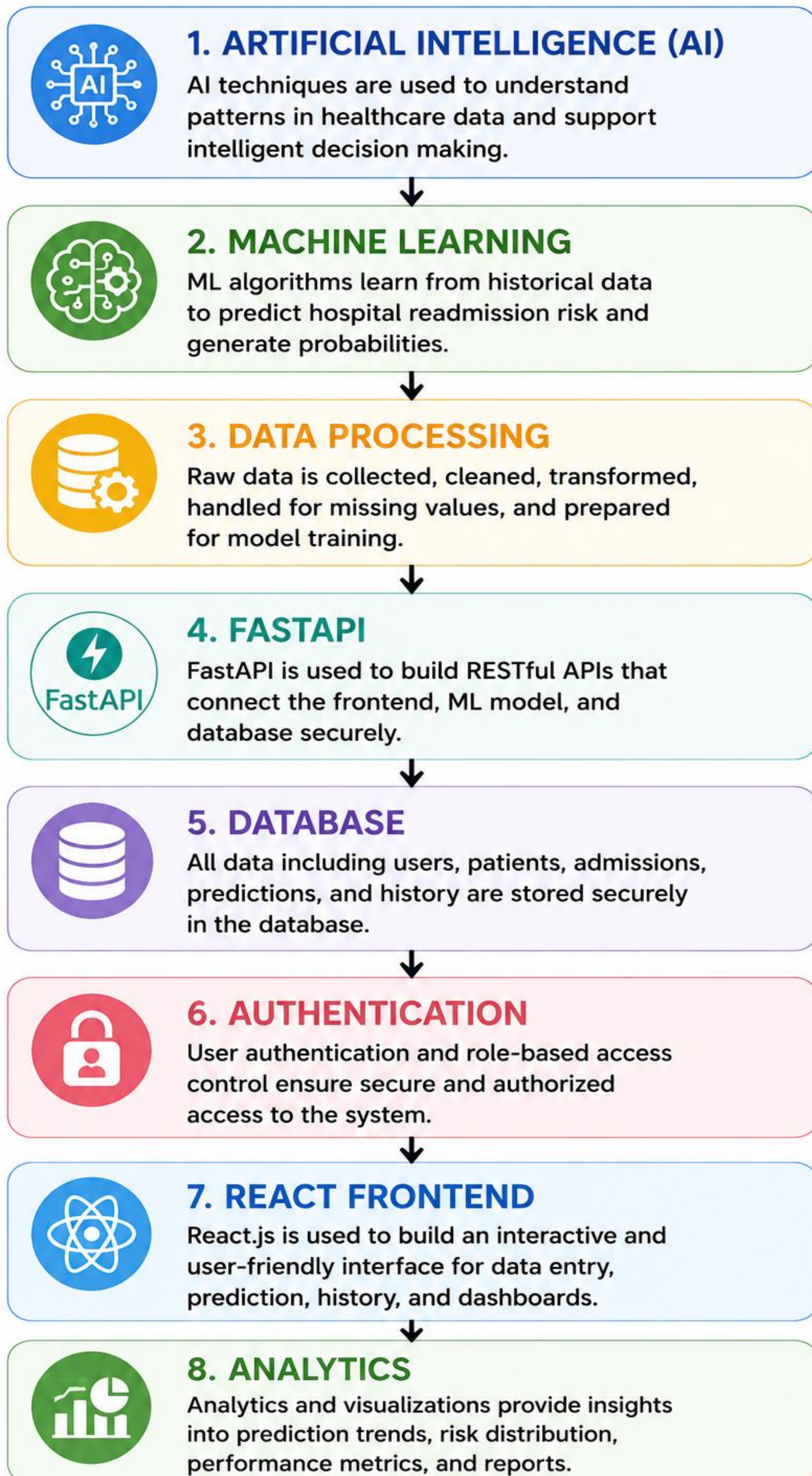
Authentication and role-based access help ensure that application functionality is available only to authorized users.

2.4.9 Scalable Architecture

The separation of frontend, backend, database, and ML components makes the application easier to maintain and extend.

2.4.10 Full-Stack AI Application

Unlike a project that only trains an ML model, this system integrates into a single practical application.



2.5 Comparison Between Existing and Proposed Systems

FEATURE	EXISTING SYSTEM	PROPOSED SYSTEM
 Patient management	✓	✓
 Admission management	✓	✓
 Manual risk assessment	✓	Supported as clinical assessment
 ML-based prediction	Limited/Not available	✓
 30-day readmission prediction	Limited	✓
 Real-time prediction	Limited	✓
 Probability-based prediction	Limited	✓
 Risk classification	Limited	✓
 Multiple healthcare data sources	Limited integration	✓
 Prediction history	Limited	✓
 Analytics dashboard	Basic/limited	✓
 Authentication	May vary	✓
 Role-based access	May vary	✓
 Automated analysis	Limited	✓
 Machine-learning model evaluation	Not typically integrated	✓

CHAPTER 3

REQUIREMENTS

3.1 Introduction

Requirements analysis is an important stage in the development of the **AI-Based Hospital Readmission Prediction System**. It identifies the functional capabilities, technical resources, software technologies, and system qualities required to develop and operate the application successfully.

The proposed system combines **Machine Learning, data processing, FastAPI, database management, authentication, React frontend, live prediction, and analytics**. Therefore, the requirements cover both the Machine Learning pipeline and the complete web application.

The requirements are divided into:

- Functional Requirements
- Non-Functional Requirements
- Hardware Requirements
- Software Requirements

3.2 Functional Requirements

Functional requirements describe the operations and services that the system must provide to its users.

3.2.1 User Authentication

The system shall provide a secure login mechanism for authorized users.

The authentication module shall:

- Accept username/email and password.
- Validate user credentials.
- Allow authenticated users to access authorized modules.
- Prevent unauthorized users from accessing protected functionality.
- Support role-based access where applicable.
- Maintain user account status.

3.2.2 Dashboard

The system shall provide a dashboard after successful login.

The dashboard shall display relevant information such as:

- Total patients
- Total admissions
- Prediction information
- Risk distribution
- Recent predictions
- Analytics information

The dashboard provides users with a quick overview of the system.

3.2.3 Patient Management

The system shall allow authorized users to manage patient information.

The module shall support:

- Adding patient records.
- Viewing patient records.
- Updating patient information.
- Searching for patients.
- Viewing patient history.
- Associating patients with admissions.

Patient information may include demographic and healthcare-related attributes required by the prediction system.

3.2.4 Admission Management

The system shall maintain hospital admission information.

The admission module shall allow authorized users to:

- Create admission records.
- View admission details.
- Associate admissions with patients.

- Store relevant admission information.
- Retrieve admission history.

Each admission can be associated with one or more prediction records.

3.2.5 Healthcare Data Integration

The system shall support the integration of information from multiple healthcare datasets.

The project uses five major relational data sources:

1. patients.csv
2. admissions.csv
3. diagnoses.csv
4. hospitals.csv
5. billing.csv

The information from these sources is combined to create a unified dataset for analysis and Machine Learning.

3.2.6 Data Preprocessing

The system shall preprocess healthcare data before model training and prediction.

The preprocessing pipeline shall handle:

- Missing values.
- Numerical variables.
- Categorical variables.
- Feature transformation.
- Numerical feature scaling.
- Categorical feature encoding.

The same preprocessing logic must be applied consistently when making live predictions.

3.2.7 Machine Learning Model

The system shall provide a Machine Learning component for predicting 30-day hospital readmission.

Multiple algorithms are evaluated during model development:

- Logistic Regression
- Random Forest
- Gradient Boosting
- XGBoost

The final implementation uses **Logistic Regression** as the primary prediction model.

3.2.8 Model Evaluation

The system development process shall evaluate the Machine Learning models using appropriate performance metrics.

The evaluation includes:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC
- Confusion Matrix

This allows different algorithms to be compared before selecting the final model.

3.2.9 Live Prediction

The system shall provide a live prediction facility.

The user can enter the required patient and admission information through the frontend. The information is sent to the FastAPI backend, where the prediction service processes the input using the trained model.

The system shall return:

- Readmission probability.
- Probability percentage.
- Risk level.
- Department information.
- Diagnosis type.

- Relevant risk factors.
- Clinical summary.
- Recommendations.

3.2.10 Risk Classification

The system shall convert the predicted probability into a simple risk category.

Risk Level	Probability Range
Low Risk	Less than 35%
Medium Risk	35%-54%
High Risk	55% or above

This classification allows users to understand the prediction without interpreting only a numerical probability.

3.2.11 Prediction History

The system shall maintain historical prediction records.

Authorized users shall be able to:

- View previous predictions.
- View associated admission information.
- View prediction probability.
- View risk classification.
- Review prediction dates.
- Track previous prediction results.

3.2.12 Database Management

The system shall store application information in a structured database.

The database shall maintain information related to:

- Users

- Departments
- Patients
- Admissions
- Predictions

Relationships between these entities shall be maintained to ensure consistent data management.

3.2.13 Analytics

The system shall provide an analytics dashboard to present useful information in a visual format.

Analytics may include:

- Risk distribution.
 - Prediction trends.
 - Patient/admission statistics.
 - High-risk patient counts.
 - Prediction-related summaries.
 - Model performance information.
-

3.2.14 API Services

The backend shall provide RESTful API services for communication between the frontend, database, and Machine Learning prediction service.

The APIs shall support operations such as:

- Authentication.
 - Patient management.
 - Admission management.
 - Prediction.
 - Prediction history.
 - Analytics.
 - Health/status checking.
-

3.3 Non-Functional Requirements

Non-functional requirements describe the quality, performance, security, usability, and maintainability characteristics of the system.

3.3.1 Performance

The system should provide prediction results within a reasonable response time after the user submits patient information.

The API should efficiently process prediction requests without unnecessary delays.

3.3.2 Security

The system should protect application and healthcare-related information.

Security requirements include:

- User authentication.
 - Password protection.
 - Role-based access.
 - Protected application functionality.
 - Controlled access to prediction and patient information.
-

3.3.3 Reliability

The system should provide consistent results when valid input is provided.

The prediction service should properly handle invalid or incomplete input rather than producing unexpected results.

3.3.4 Usability

The application should provide a simple and understandable interface.

Users should be able to:

- Log in easily.
- Navigate between modules.
- Enter patient information.
- Generate predictions.
- Understand risk categories.

- View prediction history.
 - Interpret dashboard information.
-

3.3.5 Scalability

The architecture should allow the system to be extended in the future.

Possible extensions include:

- Additional Machine Learning models.
 - Additional healthcare datasets.
 - More hospital departments.
 - Additional analytics.
 - Advanced AI capabilities.
 - Cloud deployment.
-

3.3.6 Maintainability

The application should be organized into separate components such as:

- Frontend
- Backend
- Database
- ML model
- Prediction service
- Authentication
- Analytics

This modular architecture makes the application easier to maintain and update.

3.3.7 Availability

The application should be available whenever the backend and required services are running. The FastAPI backend provides API endpoints that allow the frontend to communicate with the application services.

3.3.8 Accuracy

The Machine Learning model should be evaluated using appropriate metrics before being integrated into the prediction service.

The selected Logistic Regression model achieved:

ROC-AUC = 0.7668

on the project's time-based test dataset.

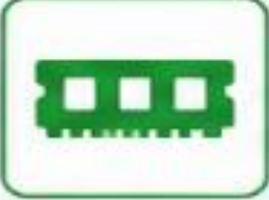
3.3.9 Compatibility

The web application should work with commonly used modern web browsers and should support communication between the React frontend and FastAPI backend through REST APIs.

3.4 Hardware Requirements

The project does not require specialized hardware for development or normal operation.

Minimum Hardware Requirements

Minimum Hardware Requirements		
Component		Requirement
	Processor	Intel Core i3 or equivalent
	RAM	8 GB minimum
	Storage	10 GB or more available space
	Display	1366 × 768 or higher
	Network	Internet connection when required
	Input	Keyboard and mouse
	Operating System	Windows/Linux/macOS

Recommended Hardware

 RECOMMENDED HARDWARE REQUIREMENTS		
COMPONENT		RECOMMENDED
	Processor	Intel Core i5/i7 or equivalent
	RAM	16 GB
	Storage	SSD
	Storage Capacity	20 GB or more
	Display	Full HD
	Network	Stable broadband connection

A GPU is **not mandatory** for the current Logistic Regression-based implementation.

3.5 Software Requirements

3.5.1 Operating System

The application can be developed and executed on:

- Windows
- Linux
- macOS

The current development environment uses **Windows with PowerShell and VS Code**.

3.5.2 Programming Language

Python

Python is used for:

- Data processing.
- Data preprocessing.
- Feature engineering.

- Machine Learning.
 - Backend development.
 - Prediction service implementation.
-

3.5.3 Backend Framework

FastAPI

FastAPI is used to develop the backend REST APIs.

It manages communication between:

React Frontend ↔ FastAPI Backend ↔ ML Model ↔ Database

It also provides API documentation through the development environment.

3.5.4 Frontend Framework

React.js

React is used to develop the interactive web interface.

The frontend provides interfaces for:

- Login.
 - Dashboard.
 - Patient management.
 - Admission management.
 - Live prediction.
 - Prediction history.
 - Analytics.
-

3.5.5 Database

A relational database is used to store application data.

The database contains entities such as:

- Users
- Departments
- Patients

- Admissions
 - Predictions
-

3.5.6 ORM

SQLAlchemy

SQLAlchemy is used to communicate between the Python backend and relational database. It provides an organized way to work with database entities and relationships.

3.5.7 Machine Learning Libraries

The project uses Python Machine Learning libraries including:

- **scikit-learn** – preprocessing, model training, evaluation.
 - **XGBoost** – XGBoost model evaluation.
 - **NumPy** – numerical operations.
 - **Pandas** – data manipulation and analysis.
 - **Joblib** – saving and loading trained model/preprocessing artifacts.
 - **SHAP** – model interpretability/feature contribution analysis where applicable.
-

3.5.8 Development Tools

The project can be developed using:

- Visual Studio Code
- Python
- PowerShell
- Git/GitHub
- Web browser
- FastAPI/Uvicorn development server

The backend can be started using:

```
uvicorn app.main:app --reload
```

The application backend runs locally during development through the Uvicorn server.

3.6 Technology Stack

The complete technology stack of the proposed system can be summarized as follows:

 TECHNOLOGY STACK		
 LAYER	 TECHNOLOGY	
 Programming Language	 Python	
 Frontend	 React.js	
 Backend	 FastAPI	
 API Server	 Uvicorn	
 Database	 Relational Database	
 ORM	 SQLAlchemy	
 ML Framework	 Scikit-learn	
 ML Algorithms	 Logistic Regression, Random Forest, Gradient Boosting, XGBoost	
 Data Processing	 pandas  NumPy Pandas, NumPy	
 Model Storage	 joblib Joblib	
 Explainability	 SHAP	
 Development Environment	 Visual Studio Code	
 Operating System	   Windows/Linux/macOS	

CHAPTER 4

SYSTEM DESIGN

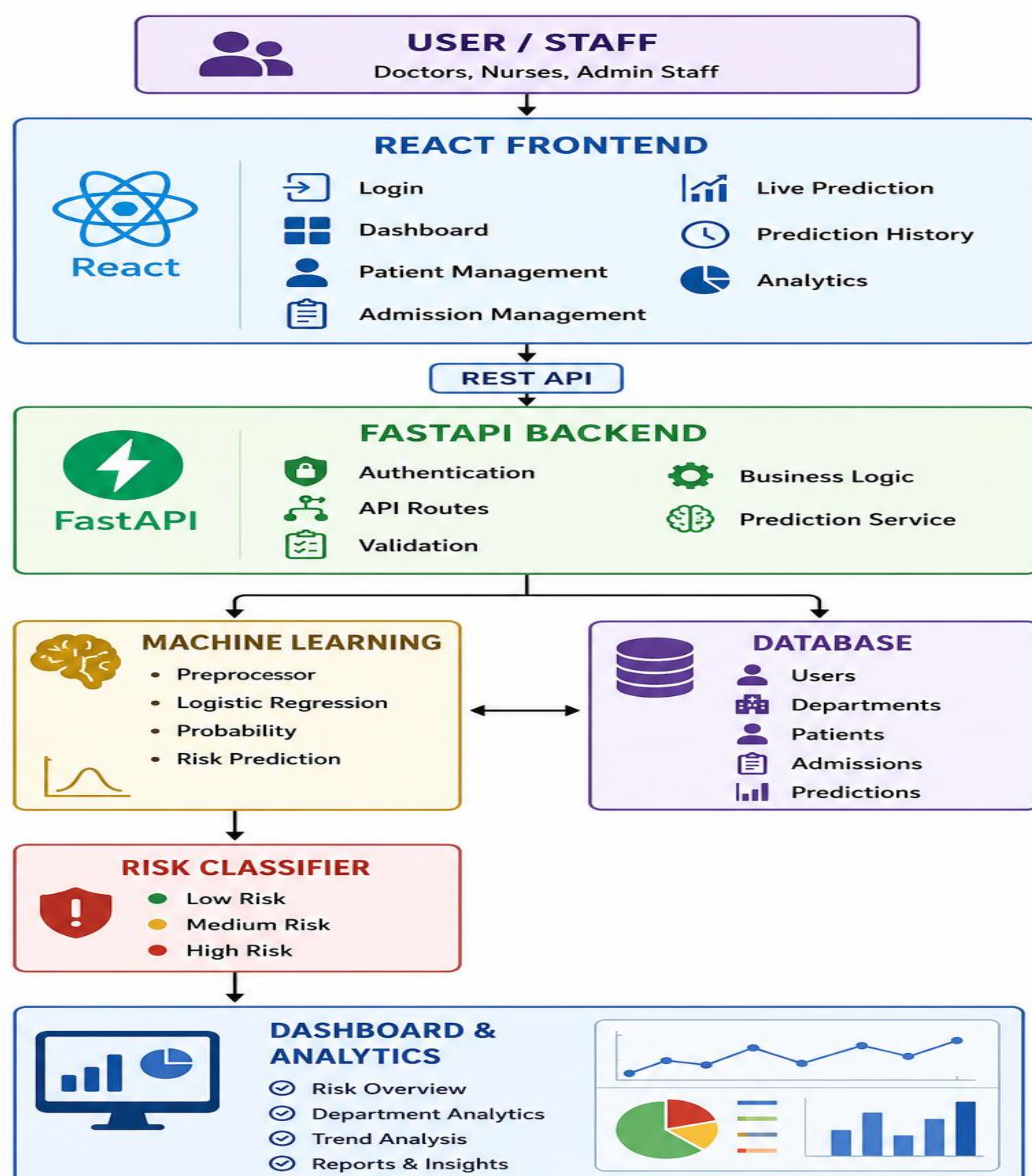
4.1 Introduction

System design is the process of defining the architecture, components, database structure, data flow, and interaction between different modules of the **AI-Based Hospital Readmission Prediction System**.

The proposed system is designed as an integrated web-based healthcare application that combines **Artificial Intelligence, Machine Learning, data processing, FastAPI, database management, authentication, React frontend, live prediction, and analytics**.

The system follows a layered architecture in which the frontend communicates with the backend through APIs. The backend manages authentication, patient and admission information, database operations, and communication with the trained Machine Learning model. The prediction service processes user input and generates the probability of 30-day hospital readmission.

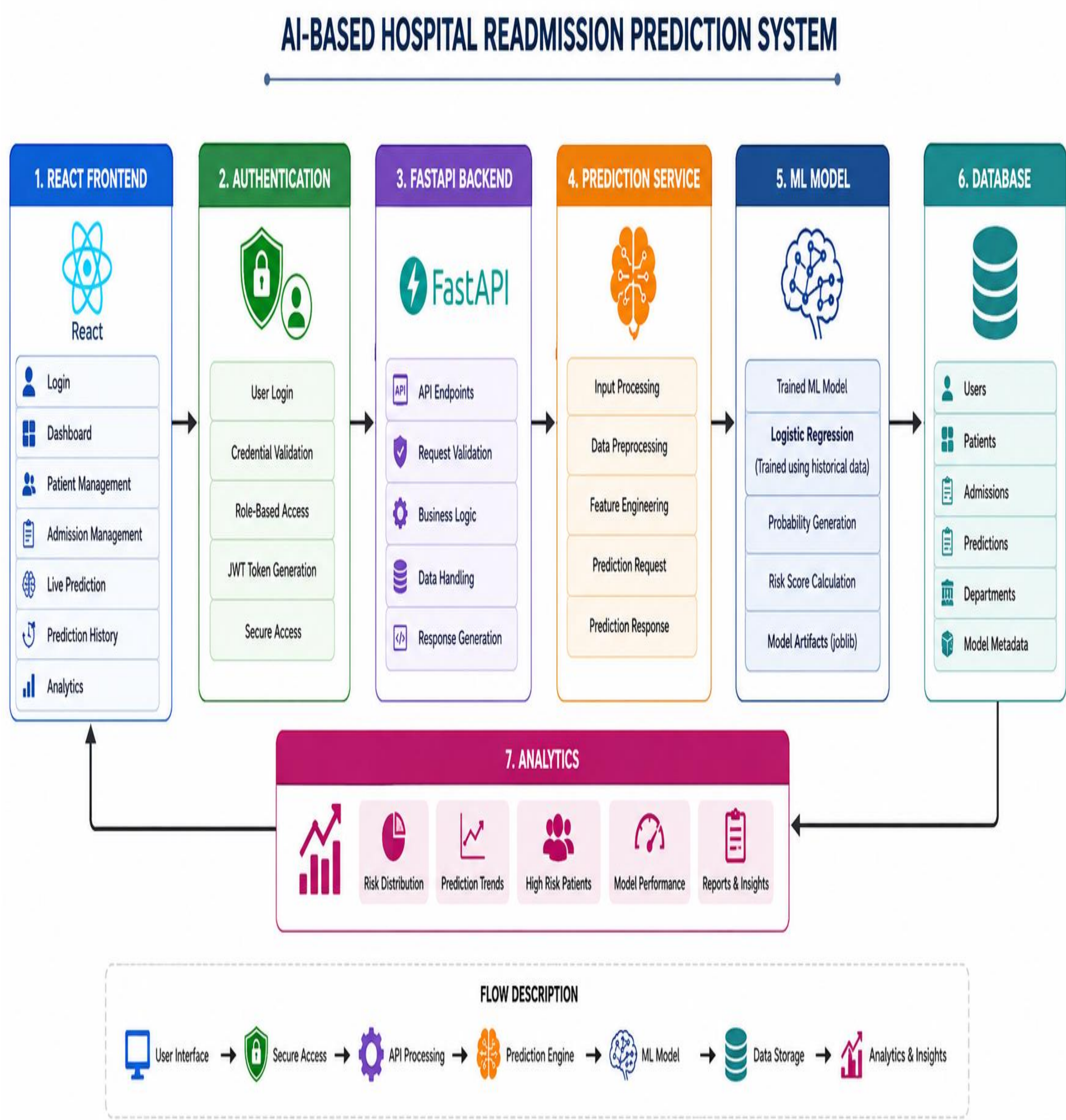
The major system components are:



4.2 System Architecture

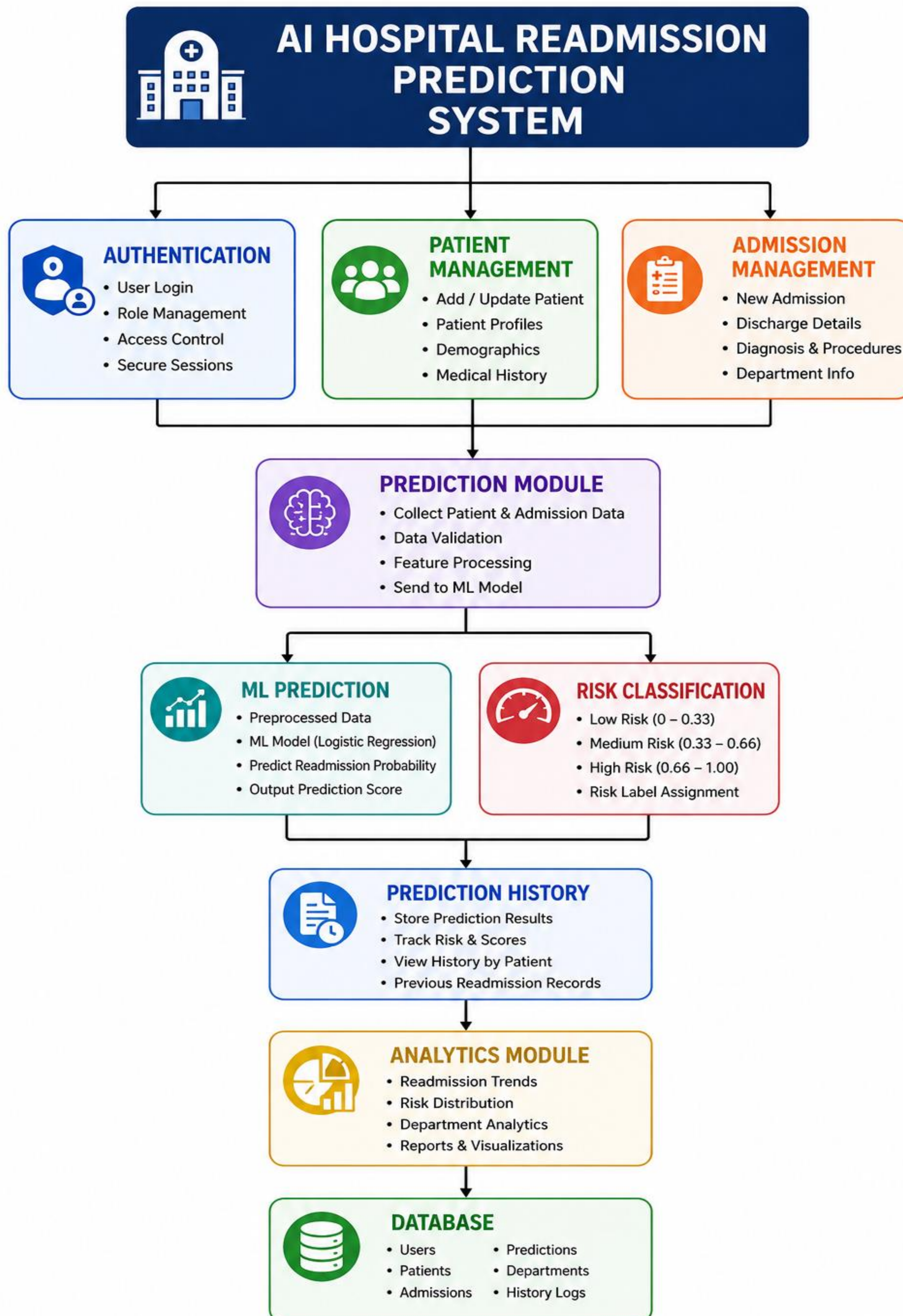
The system architecture describes how the major components of the application interact with one another.

4.2.1 High-Level Architecture



4.3 Module Architecture

The proposed system is divided into independent but interconnected modules.



4.3.1 Authentication Module

The authentication module controls access to the application.

Functions

- User login.
- Credential validation.
- User authorization.
- Role-based access.
- Active/inactive account management.
- Protection of application modules.

Only authenticated and authorized users should be able to access protected system functionality.

4.3.2 Patient Management Module

This module manages patient-related information.

Functions

- Add patient.
 - View patient.
 - Update patient information.
 - Search patient.
 - View patient history.
 - Associate patients with admissions.
-

4.3.3 Admission Management Module

This module maintains hospital admission information.

Functions

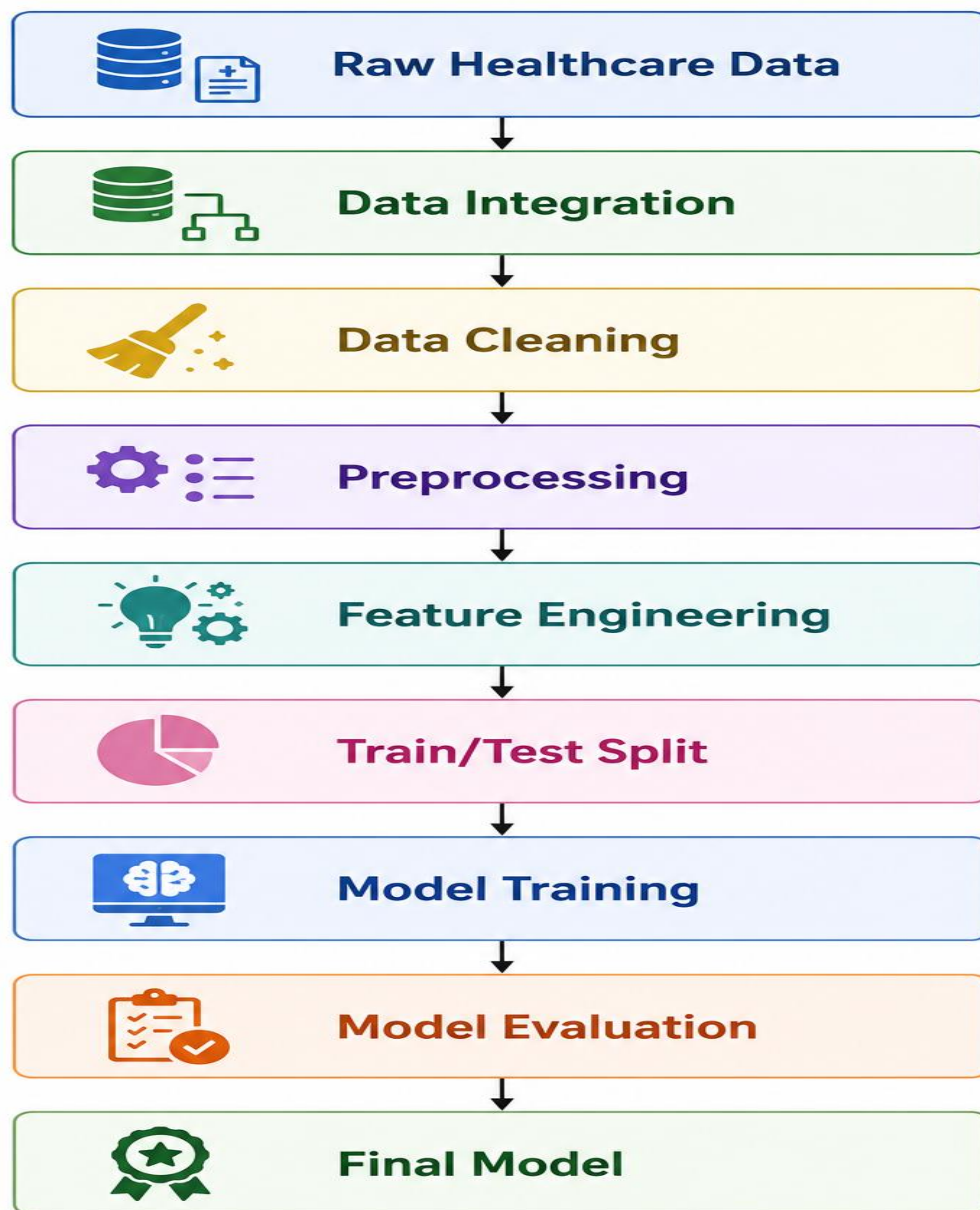
- Create admission.
- View admission.
- Associate admission with a patient.
- Store admission-related information.
- Retrieve admission history.

The admission record acts as an important reference for the readmission prediction process.

4.3.4 Machine Learning Module

The Machine Learning module is the core analytical component of the system.

The development process includes:



The project evaluates:

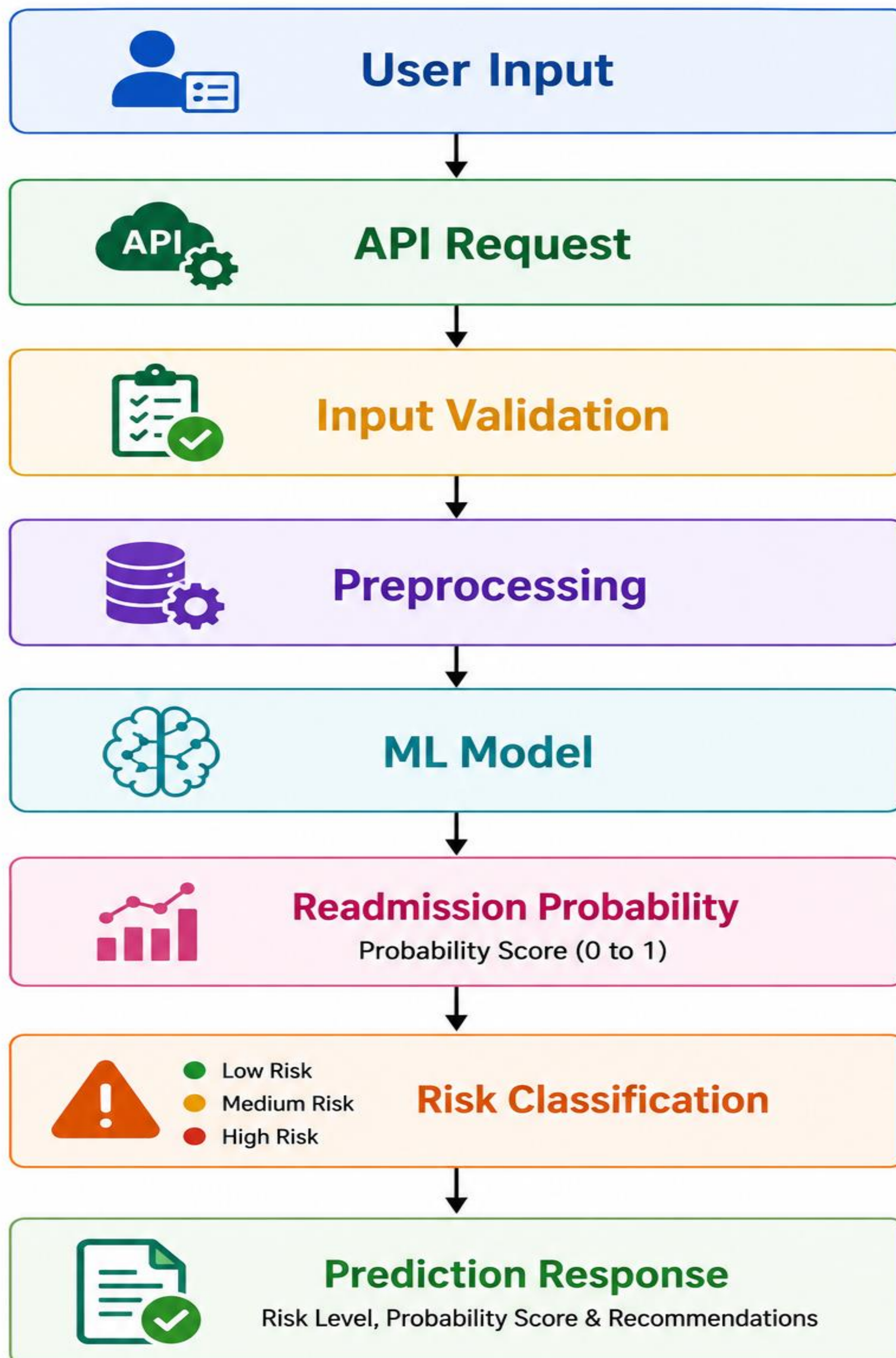
- Logistic Regression
- Random Forest
- Gradient Boosting
- XGBoost

The final selected model is **Logistic Regression**.

4.3.5 Prediction Module

The prediction module performs live readmission-risk prediction.

The process is:



The model generates a probability representing the estimated likelihood of 30-day readmission.

4.3.6 Risk Classification Module

The risk classification module converts the predicted probability into an understandable risk category.

Risk Level	Probability Range
Low Risk	Less than 35%
Medium Risk	35%-54%
High Risk	55% or above

This makes the ML output easier to interpret for application users.

4.3.7 Prediction History Module

This module stores and displays previous prediction results.

It can maintain information such as:

- Admission ID
- Prediction probability
- Risk level
- Department
- Diagnosis type
- Prediction date
- Model information

This allows users to review previous predictions.

4.3.8 Analytics Module

The analytics module transforms stored information into useful summaries and visualizations.

It can provide:

- Total patient statistics.
- Admission statistics.
- Risk distribution.
- High-risk patient information.

- Prediction trends.
- Model performance information.

4.4 Database Design

The database is designed to store structured application information and maintain relationships between users, departments, patients, admissions, and predictions.

Main entities

1. **Users**
2. **Departments**
3. **Patients**
4. **Admissions**
5. **Predictions**

4.4.1 Users Table

Field	Description
user_id	Unique user identifier
username/email	User login information
password_hash	Encrypted/hashed password
role	User role
department	Associated department
designation	User designation
is_active	Account status

4.4.2 Admissions Table

Field	Description
admission_id	Unique admission identifier
patient_id	Associated patient
hospital information	Hoapital-related details
admission date	Date of admission
discharge date	Date of discharge
diagnosis information	Diagnosis-related information
department_id	Associated department

4.4.3 Departments Table

Field	Description
Department_id	Unique department identifier
Department_name	Name of hospital department
Description	Department description
specialization	Department specialization

4.4.4 Patients Table

Field	Description
patient_id	Unique patient identifier
patient details	Demographic information
gender	Patient gender
age	Patient gender
state	Patient state
insurance_type	Insurance information
bpl_card	BPL-card information
comorbidity_count	Number of comorbidities
prev_admissions	Previous admissions

4.4.5 Predictions Table

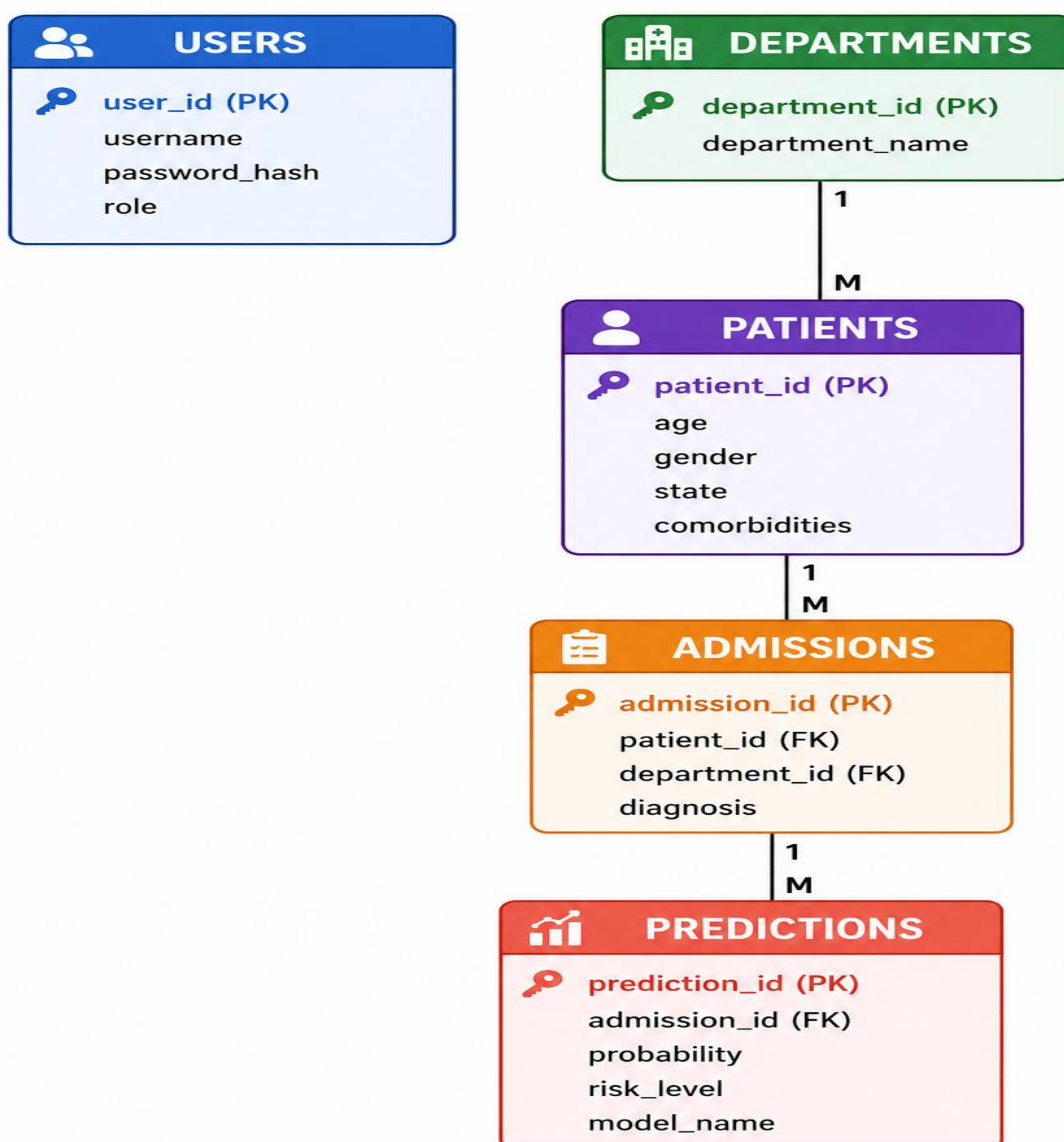
Field	Description
prediction_id	Unique prediction identifier
admission_id	Associated admission
probability	Predicted probability
probability_percent	Probability in percentage
risk_level	Low/Medium/High
diagnosis_type	Type of diagnosis
risk_factors	Identified risk factors
clinical_summary	Prediction summary
recommendations	Suggested follow-up information
model_name	ML model used
created_at	Prediction timestamp

4.5 Entity Relationship Diagram

The major relationships between database entities are:

Relationships

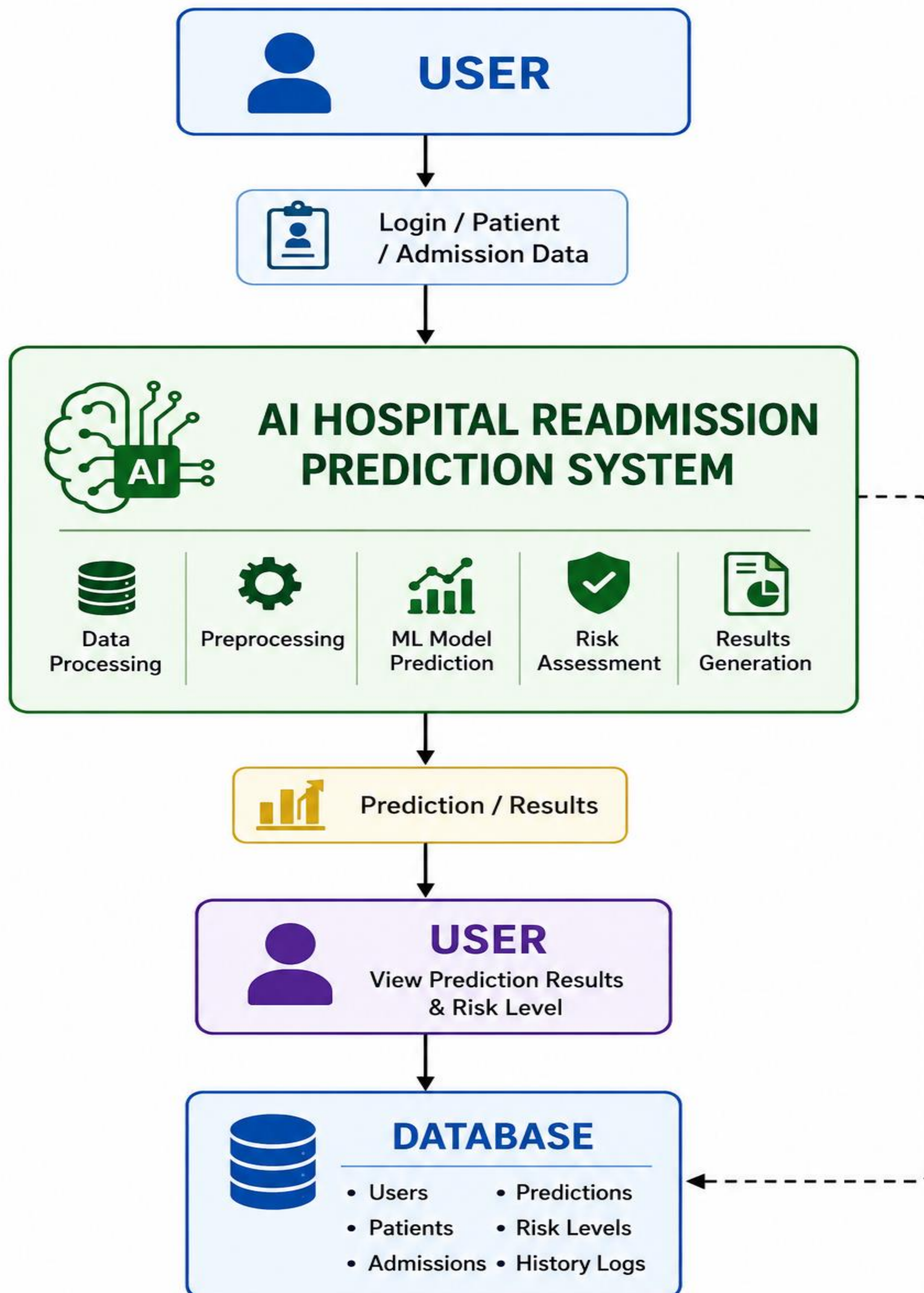
- One department can be associated with multiple patients/admissions.
- One patient can have multiple admissions.
- One admission can have multiple prediction records.
- Each prediction is associated with a particular admission.



4.6 Data Flow Diagram

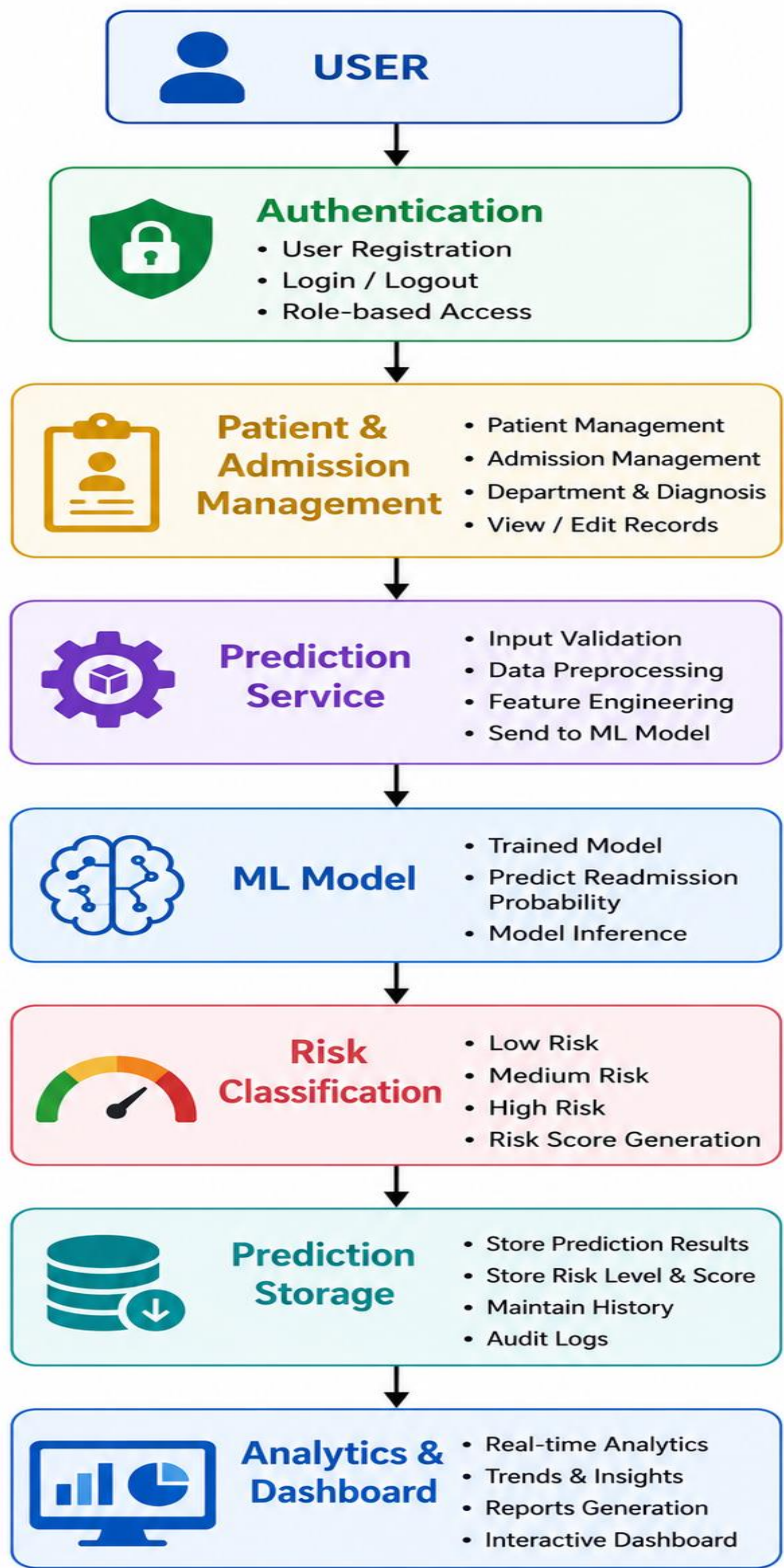
4.6.1 Level 0 DFD

The Level 0 Data Flow Diagram represents the system as a single process.



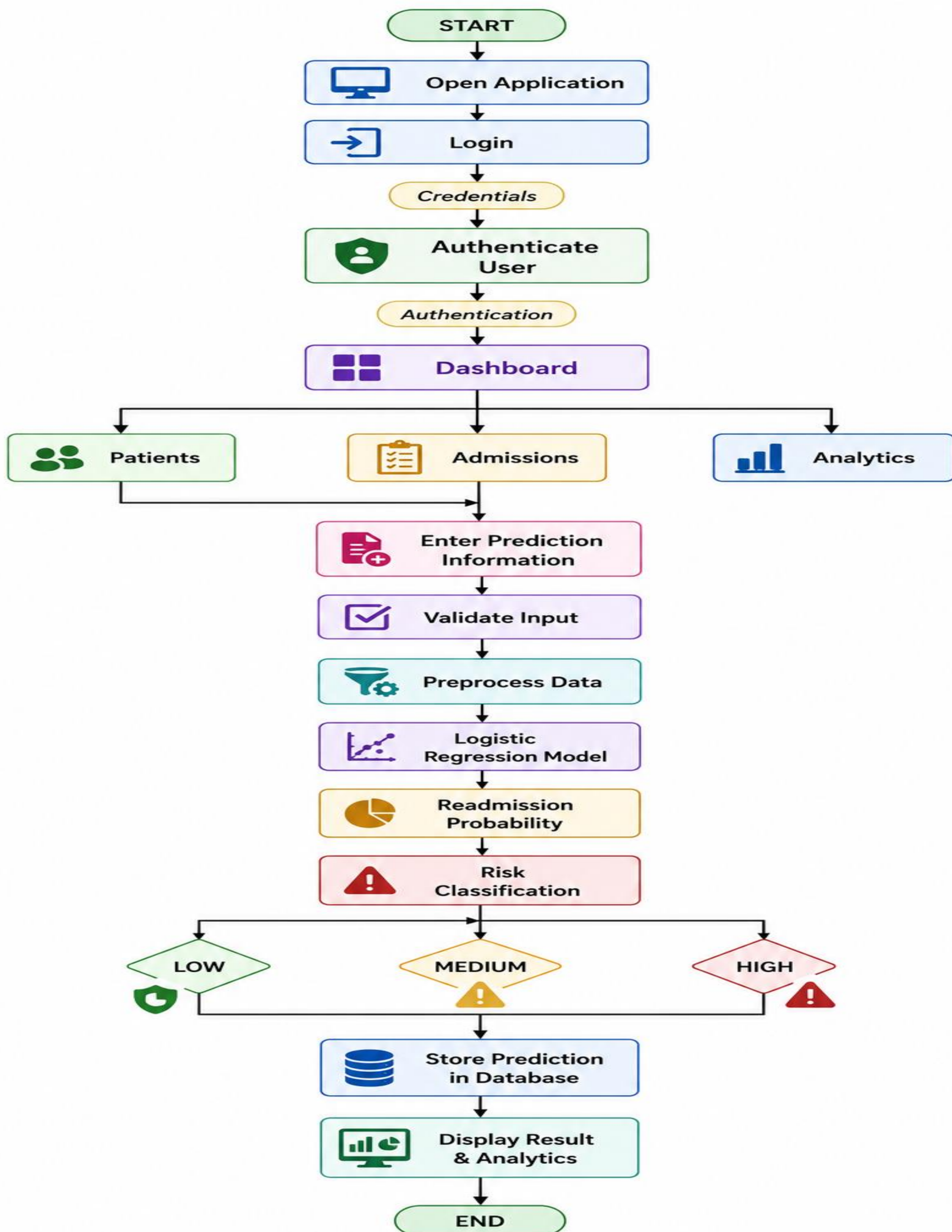
4.6.2 Level 1 DFD

The Level 1 DFD expands the system into its major processes.



4.7 System Flow

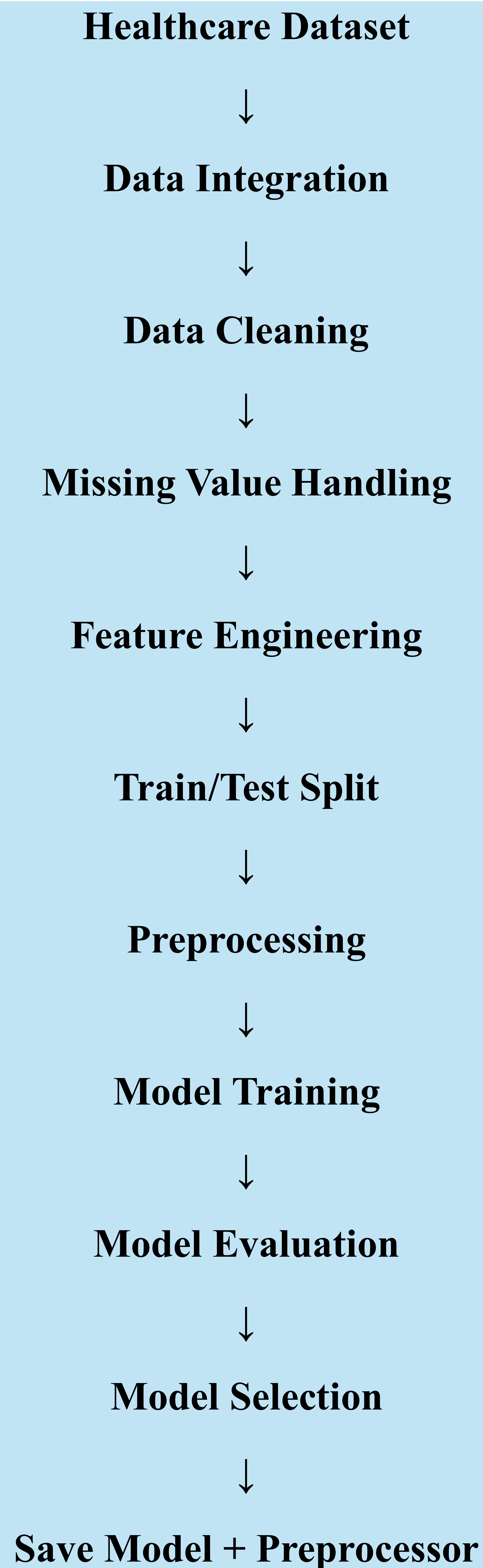
The complete system flow describes how a user interacts with the application and how the prediction is generated.



4.8 Machine Learning Design

The Machine Learning component follows a separate training and prediction pipeline.

4.8.1 Training Pipeline



The project's data uses a **time-based split**, where historical data is used for training and a later period is used for testing. This provides a more realistic assessment of model performance on future-like data.

4.8.2 Prediction Pipeline

New Patient Information

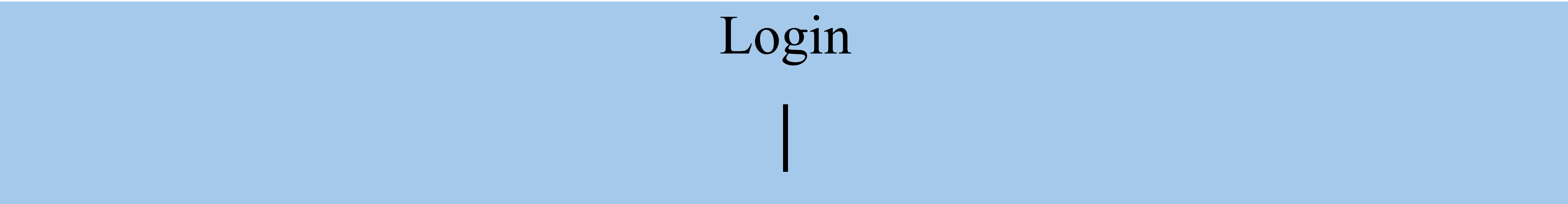


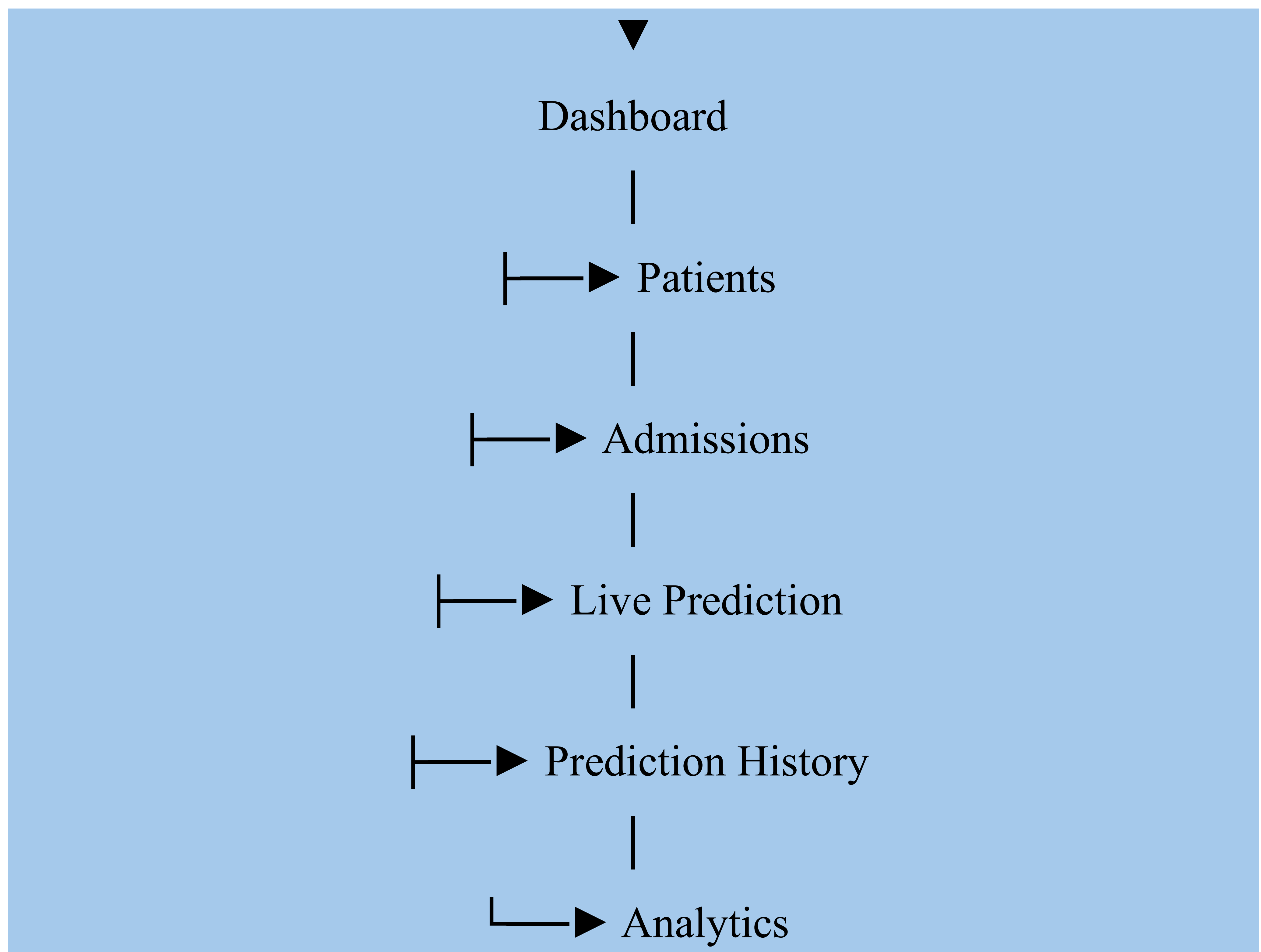


4.9 User Interaction Design

The application provides a structured navigation flow for authenticated users.

Main navigation





The interface is designed so that users can move from patient information to admission details and then to live prediction without manually transferring information between separate applications.

4.10 Prediction Result Design

The prediction result is designed to provide both numerical and descriptive information.

Example result structure

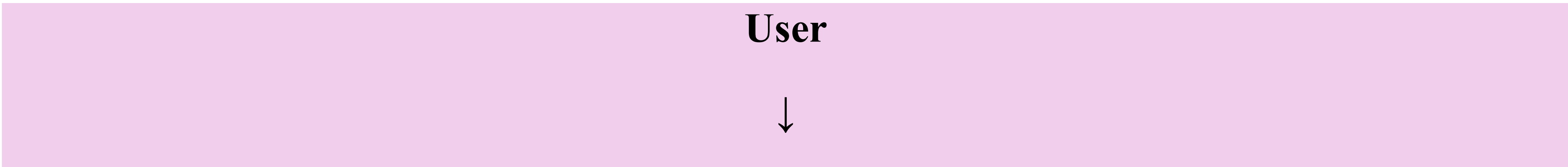
This design allows users to understand the prediction without requiring knowledge of Machine Learning.

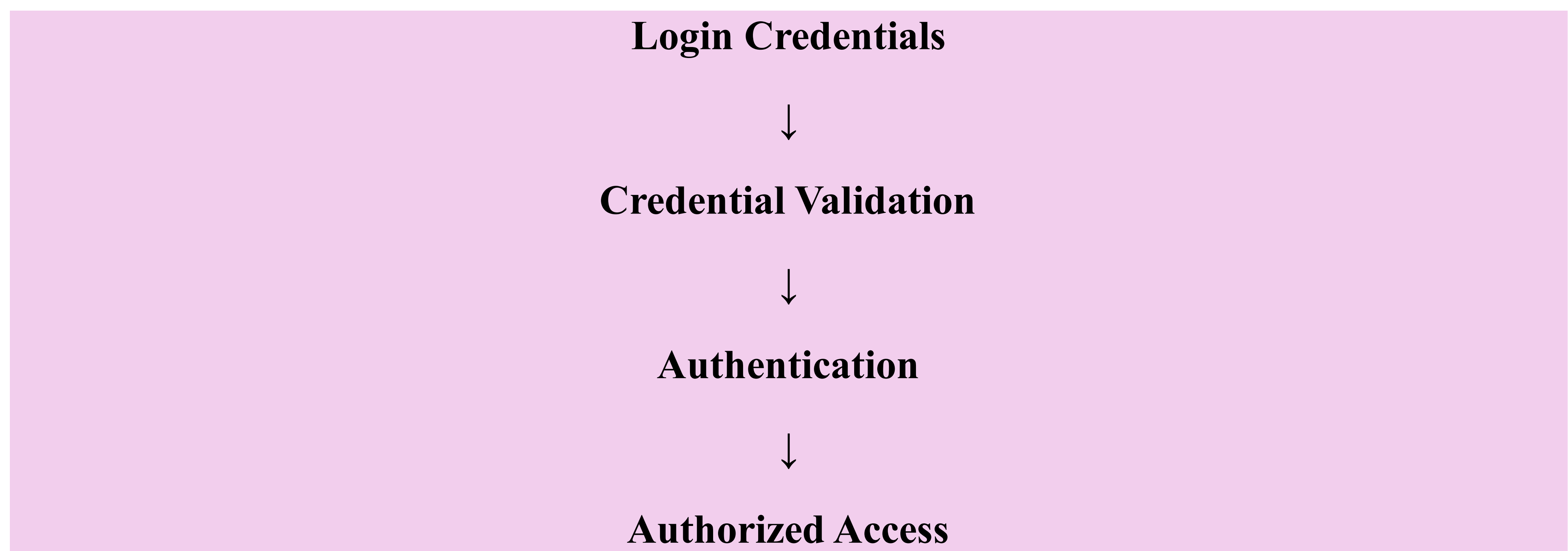
	Admission ID: ADM-XXXX	
	Probability: 68.40%	
	Risk Level: HIGH	
	Department: Cardiology	
	Diagnosis Type: Chronic Condition	
	Risk Factors:	
	• Previous admissions	
	• Comorbidities	
	• Diagnosis-related factors	
	Clinical Summary:	
	Patient has an elevated predicted	
	probability of readmission.	
	Recommendations:	
	Appropriate follow-up and	
	monitoring may be considered.	

4.11 Security Design

Security is considered at different levels of the application.

Authentication





Access Control

Users are provided access according to their assigned role and permissions.

Data Protection

The system should protect:

- User credentials.
- Patient information.
- Admission information.
- Prediction records.
- Database access.

Passwords are maintained using a secure password-hashing mechanism rather than storing them as plain text.

4.12 Chapter Summary

This chapter presented the design of the **AI-Based Hospital Readmission Prediction System**. The system architecture, module structure, database design, entity relationships, data flow, system flow, Machine Learning pipeline, prediction workflow, user interaction, and security design were described.

The proposed design integrates:

React Frontend → FastAPI Backend → Authentication → Database → ML Prediction Service → Risk Classification → Analytics

CHAPTER 5

IMPLEMENTATION

5.1 Introduction

Implementation is the stage in which the system design is converted into a working software application. The **AI-Based Hospital Readmission Prediction System** is implemented as an integrated web-based application combining a React frontend, FastAPI backend, relational database, authentication mechanism, and Machine Learning prediction service.

The implementation is divided into several major components:

- Frontend implementation
- Backend implementation
- Database implementation
- Authentication implementation
- Machine Learning implementation
- Prediction service implementation
- Analytics implementation

The overall implementation flow is:

React Frontend → Authentication → FastAPI Backend → Prediction Service → ML Model → Database → Analytics

The system also supports **live prediction**, where newly entered patient and admission information is processed through the trained Machine Learning pipeline and a readmission-risk result is returned to the user.

5.2 Frontend Implementation

The frontend of the application is developed using **React.js**. It provides an interactive interface through which authorized users can access different modules of the hospital readmission prediction system.

The frontend is responsible for:

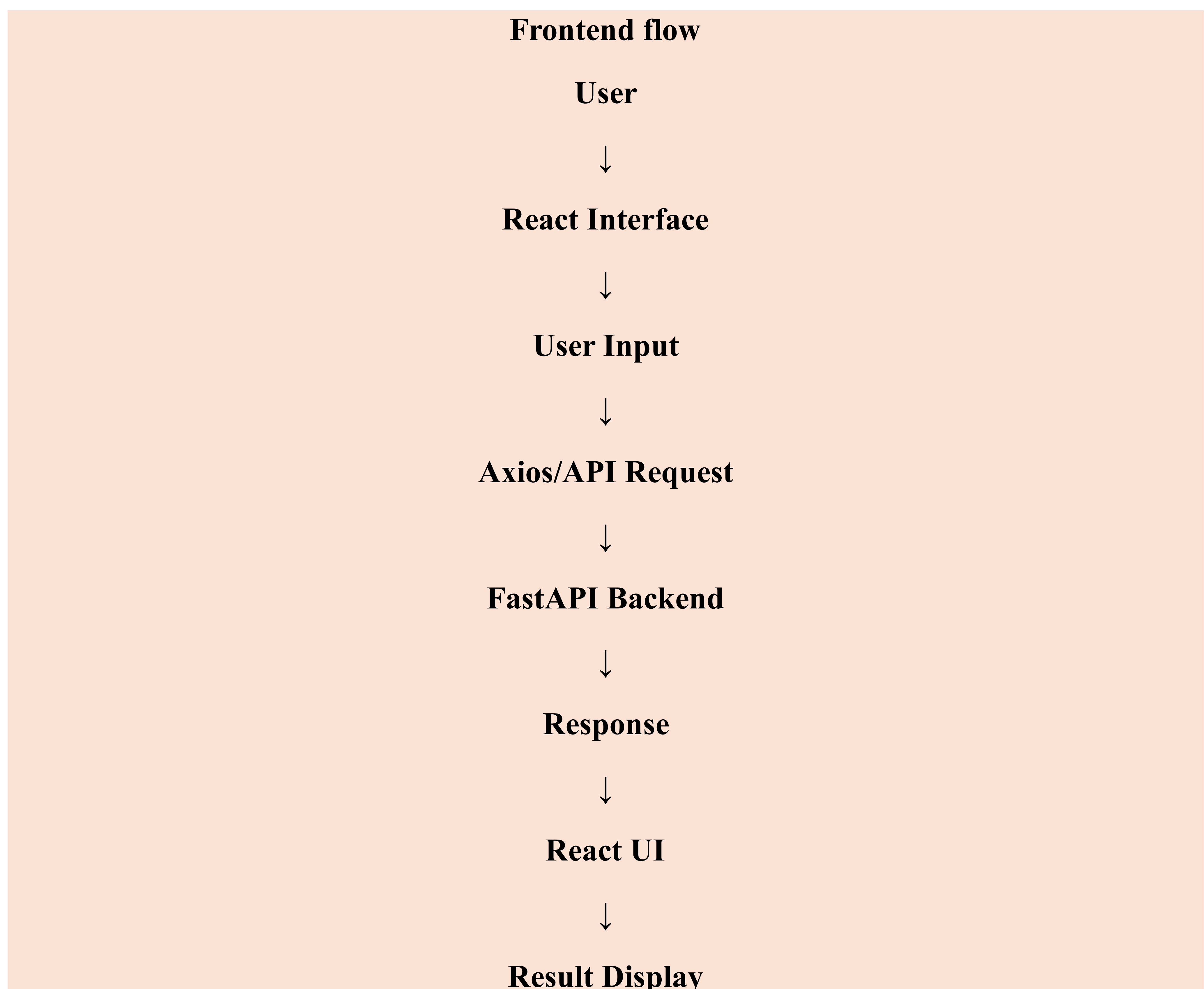
- Displaying application pages.
- Collecting user input.
- Sending requests to backend APIs.
- Displaying API responses.

- Presenting prediction results.
- Displaying patient and admission information.
- Showing prediction history.
- Presenting analytics.

Major frontend screens

The application contains interfaces for:

1. Login
2. Dashboard
3. Patient Management
4. Admission Management
5. Live Prediction
6. Prediction History
7. Analytics

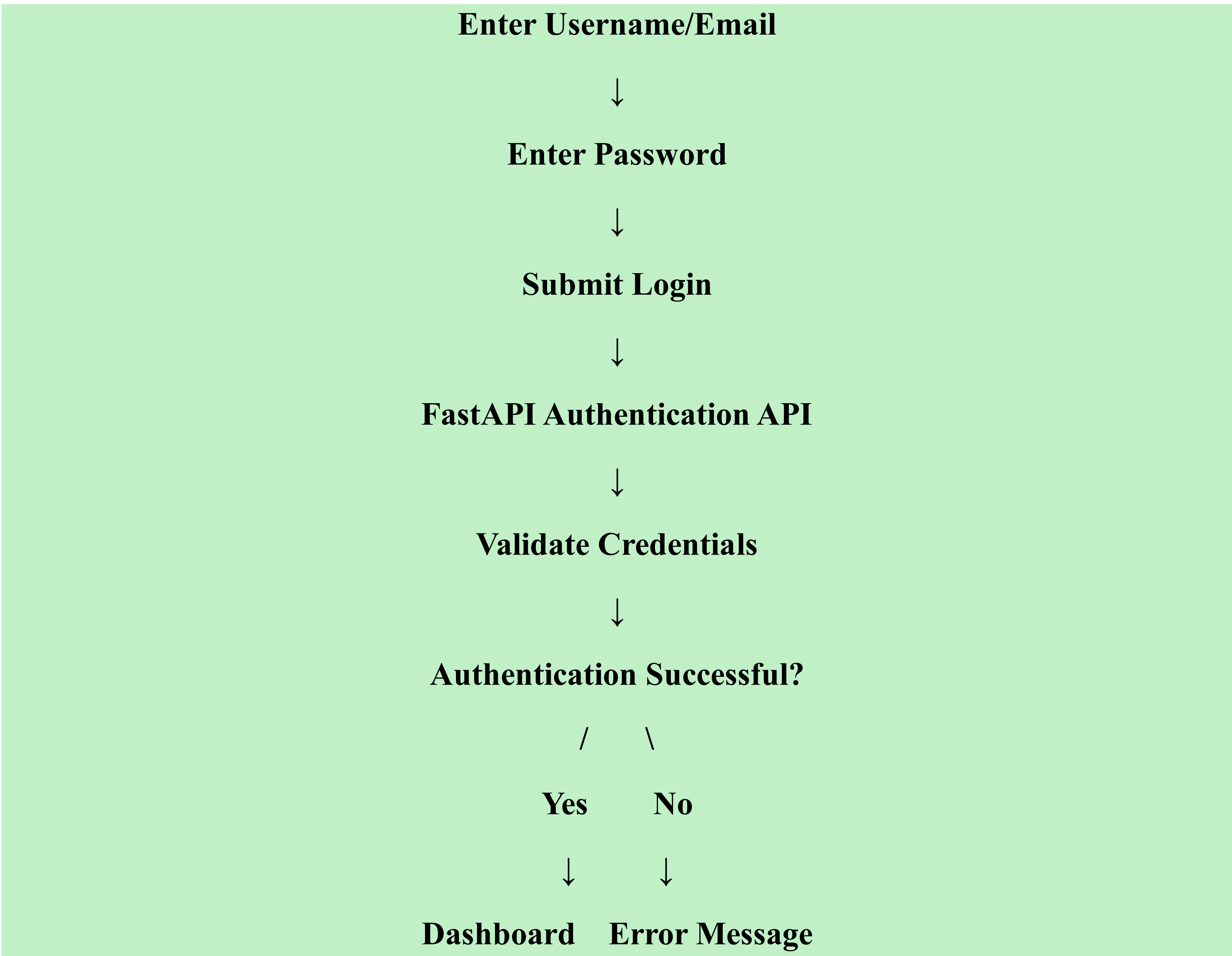


React components are organized so that individual application features can be developed and maintained independently.

5.2.1 Login Interface

The login page provides an interface for authorized users to enter their credentials.

The login process is:



The frontend displays an appropriate error message when authentication fails.

5.2.2 Dashboard Implementation

The dashboard acts as the main interface after successful authentication.

It provides an overview of system information such as:

- Total patients.
- Total admissions.
- Prediction statistics.

- Risk distribution.
- Recent prediction information.
- Analytics.

The dashboard allows users to quickly access the main modules of the application.

5.2.3 Patient Management Interface

The patient management interface provides functionality for maintaining patient records.

The user can:

- Add a patient.
- View patient details.
- Search for a patient.
- Update patient information.
- View associated admissions.

Patient information used by the application may include demographic and historical attributes required for prediction.

5.2.4 Admission Management Interface

The admission interface manages hospital admission records.

The user can:

- Create admission records.
- View admission information.
- Associate an admission with a patient.
- View admission history.
- Initiate prediction for an admission.

The admission ID provides an important connection between the patient record and prediction record.

5.3 Backend Implementation

The backend is developed using **Python and FastAPI**.

FastAPI provides REST APIs that connect the React frontend with the application's business logic, database, and Machine Learning prediction service.

The backend is responsible for:

- Request handling.
- Input validation.
- Authentication.
- Patient management.
- Admission management.
- Prediction requests.
- Model loading.
- Prediction generation.
- Database operations.
- Analytics services.

5.3.1 FastAPI Application

The main FastAPI application initializes the backend and connects the different API modules.

The backend provides endpoints for functions such as:

- Authentication.
- Patient management.
- Admission management.
- Prediction.
- Prediction history.
- Analytics.
- Health checking.

During development, the backend can be started using:

```
uvicorn app.main:app --reload
```

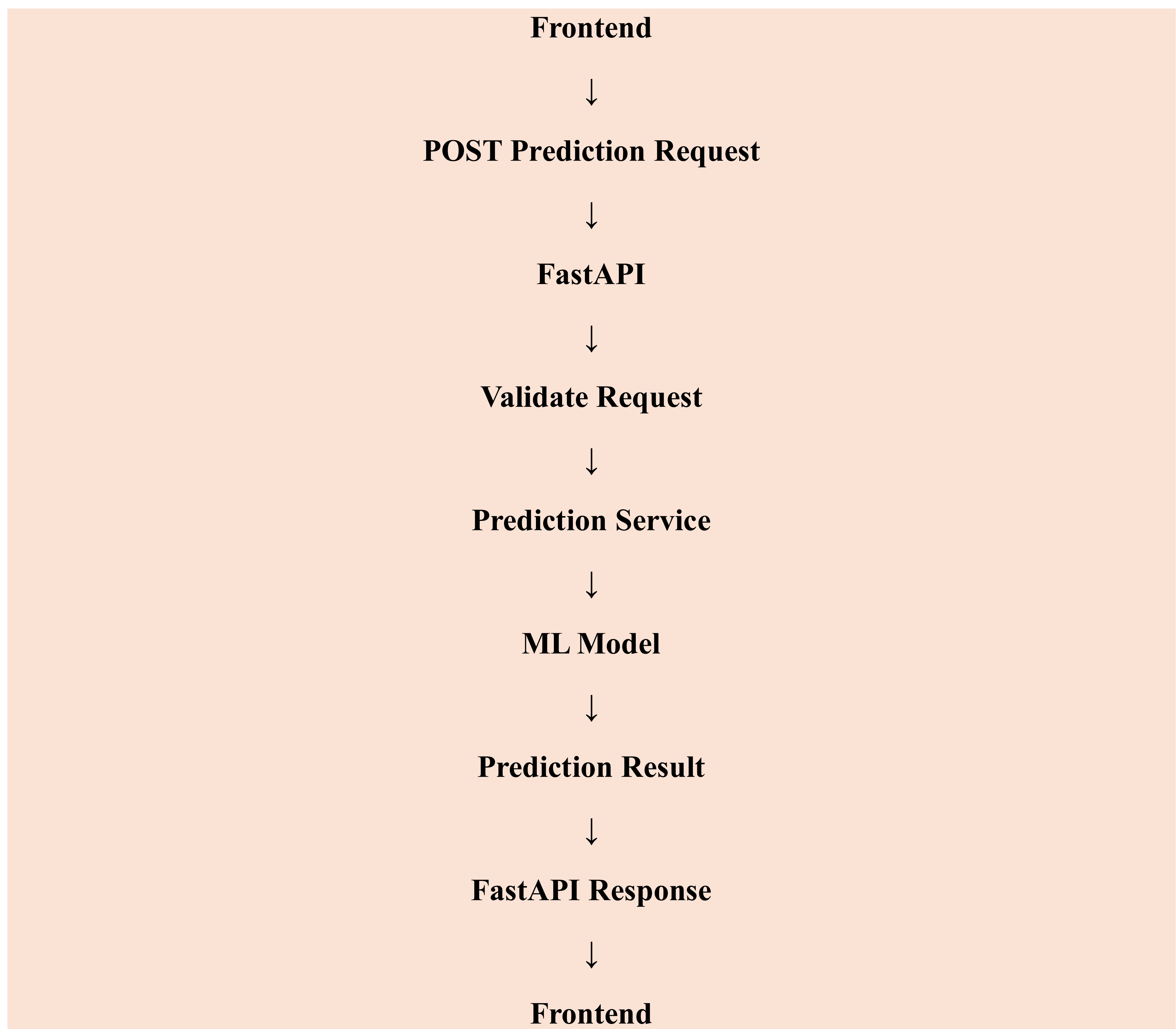
The API documentation can be accessed through the FastAPI documentation interface during local development.

Backend structure

A simplified structure of the application is:

```
backend/
|
|— app/
|   |— main.py
|   |— database.py
|   |— models.py
|   |
|   |— api/
|       |— prediction.py
|       |
|       |— schemas/
|           |— prediction.py
|           |
|           |— services/
|               |— prediction_service.py
|               |
|               |— trained_models/
|                   |— preprocessor.joblib
|                   |— trained model
|                   |
|                   |— scripts/
|                       |— inspect_dataset.py
|                       |— build_training_data.py
|                       |— analyze_time_split.py
|                       |— train_models.py
|                       |— analyze_thresholds.py
```


5.3.2 API Request and Response: The prediction API follows the general flow:



The response contains information such as:

- Admission ID
- Probability
- Probability percentage
- Risk level
- Department name
- Diagnosis type
- ICD-10 code
- Risk factors
- Clinical summary
- Recommendations

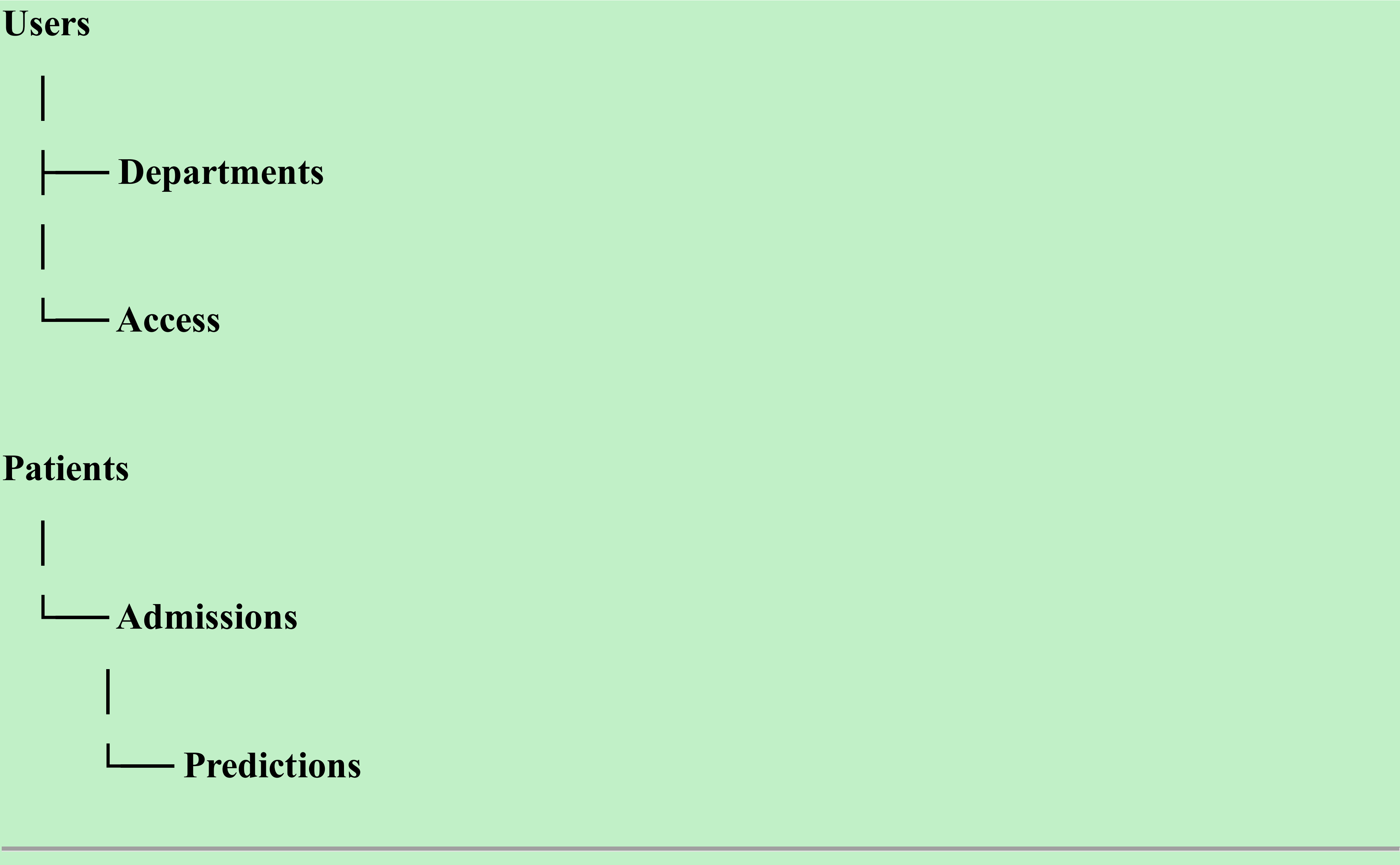
- Model name
- Model version

5.4 Database Implementation

A relational database is used to store the application's structured information.

SQLAlchemy is used as the Object Relational Mapper (ORM) to allow the Python backend to interact with database tables using Python objects and models.

The major entities include:



5.4.1 Users

The Users entity stores information required for authentication and authorization.

Important attributes include:

- User ID
- Username/email
- Password hash
- Role
- Department
- Designation

- Account status
-

5.4.2 Departments

The Departments entity stores hospital department information.

Examples may include:

- Cardiology
- Neurology
- Orthopedics
- General Medicine
- Gynecology

The department information can be associated with patients or admissions according to the application's database design.

5.4.3 Patients

The Patients entity stores patient-related information.

Important attributes include:

- Patient ID
 - Age
 - Gender
 - State
 - BPL card information
 - Insurance type
 - Comorbidity count
 - Previous admissions
-

5.4.4 Admissions

The Admissions entity stores information about individual hospital admissions.

An admission is associated with a particular patient and may contain:

- Admission ID

- Patient ID
 - Department
 - Diagnosis information
 - Admission details
 - Discharge information
 - Hospital information
-

5.4.5 Predictions

The Predictions entity stores the output generated by the Machine Learning system.

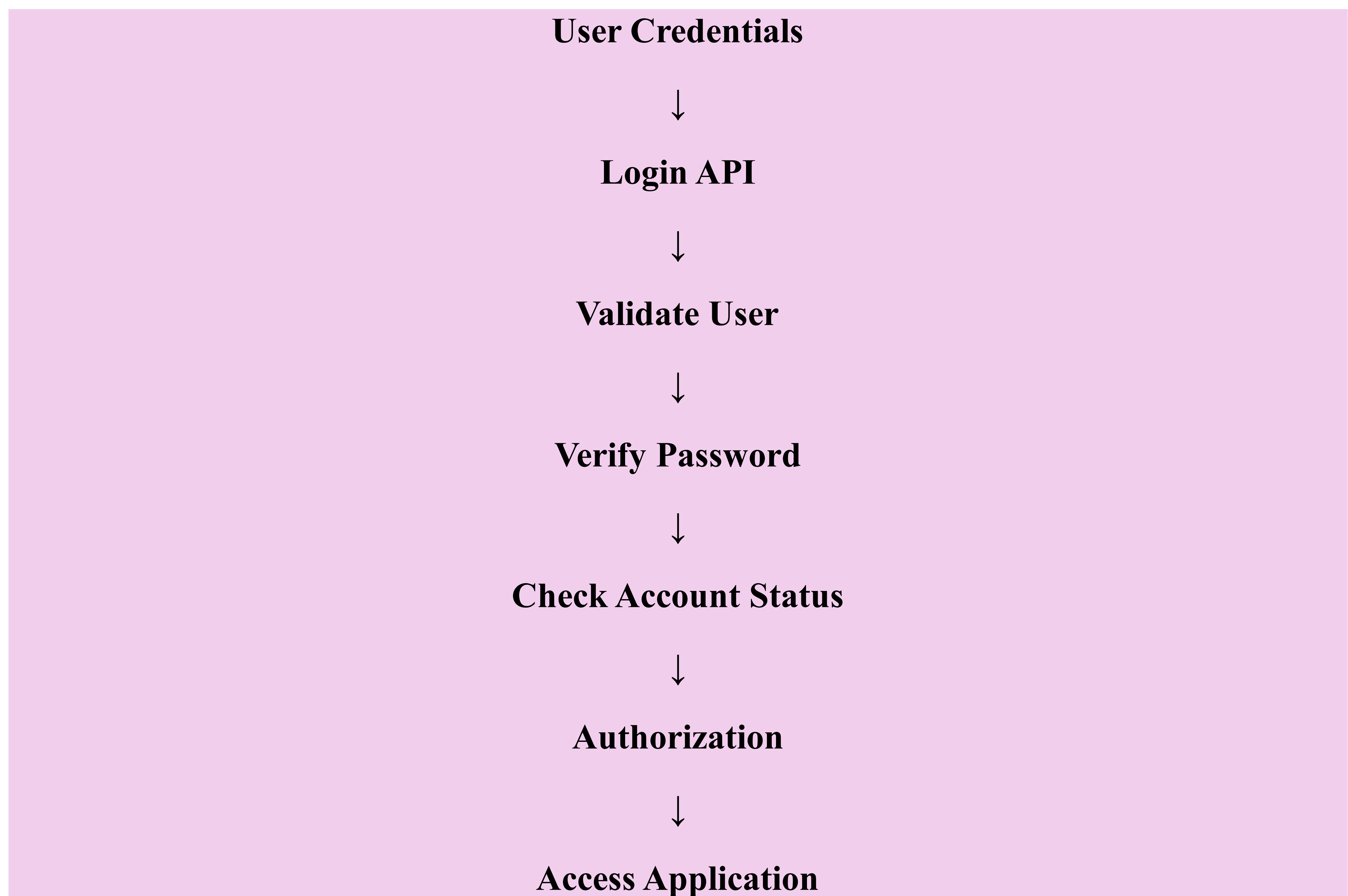
It can contain:

- Prediction ID
 - Admission ID
 - Probability
 - Risk level
 - Diagnosis type
 - Risk factors
 - Clinical summary
 - Recommendations
 - Model name
 - Model version
 - Prediction timestamp
-

5.5 Authentication Implementation

Authentication is implemented to ensure that only authorized users can access protected system functionality.

The authentication process begins when the user enters login credentials.



Passwords should not be stored as plain text. Instead, password-hashing mechanisms are used to protect stored credentials.

Authentication provides an additional security layer for patient information and prediction records.

5.6 Machine Learning Implementation

Machine Learning is the core analytical component of the project.

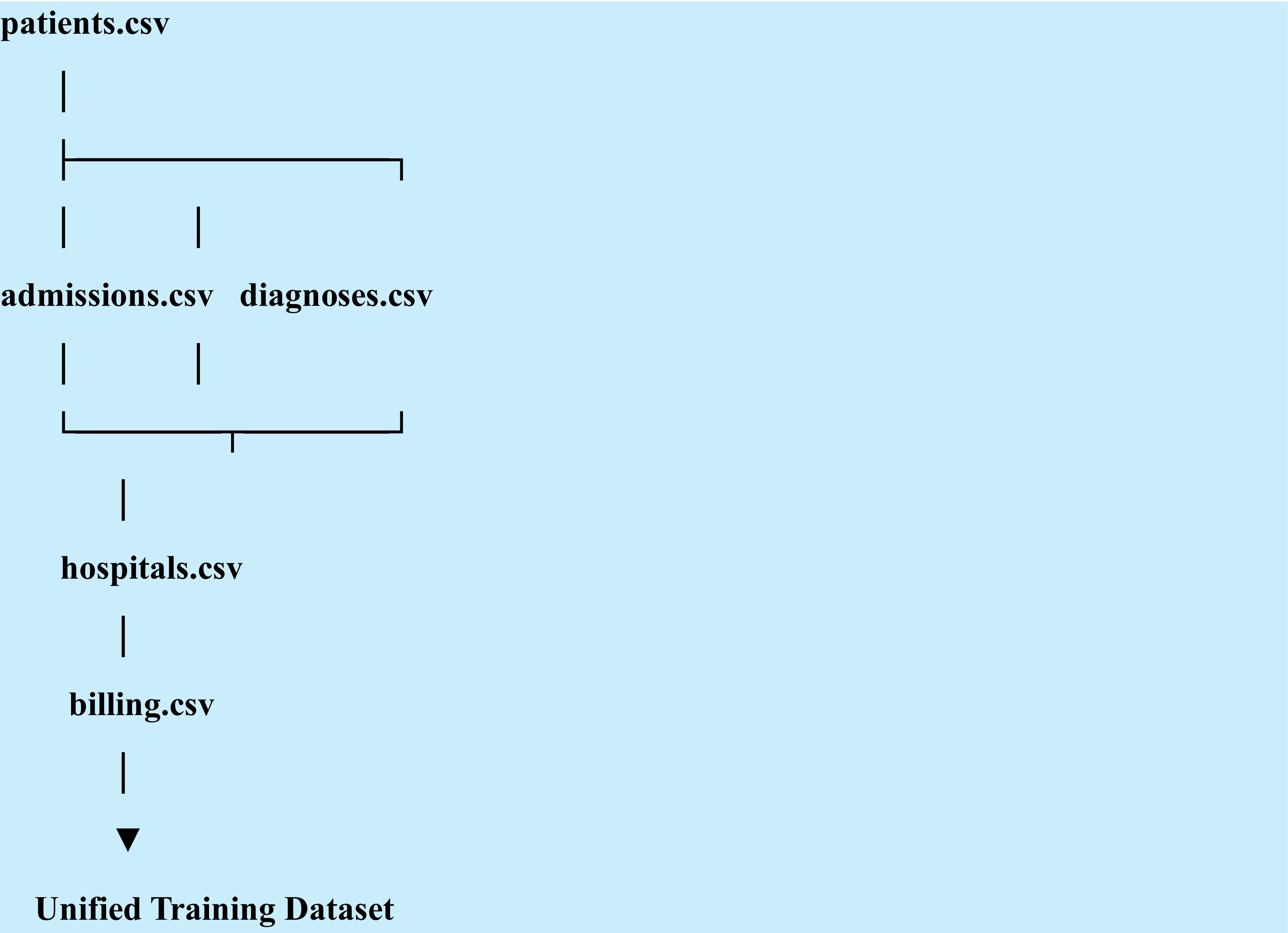
The ML implementation consists of:

1. Dataset inspection
2. Data integration
3. Data preprocessing
4. Feature engineering
5. Time-based data splitting
6. Model training
7. Model evaluation
8. Model selection
9. Model persistence

10. Live prediction

5.6.1 Dataset Integration

The project uses five healthcare datasets:



The datasets are combined using appropriate identifiers and relationships.

The resulting integrated dataset contains approximately **120,000 admission records** and **36 columns**.

5.6.2 Data Preprocessing

The raw healthcare data contains both numerical and categorical variables.

Numerical preprocessing

Numerical variables are processed using:

- Missing-value imputation.
- Median replacement where required.
- Feature scaling.

Categorical preprocessing

Categorical variables are processed using:

- Missing-value imputation.
- Most-frequent value replacement.
- One-hot encoding.

The preprocessing pipeline is saved so that the same transformations can be applied to live prediction inputs.

5.6.3 Feature Engineering

Feature engineering prepares meaningful variables for the Machine Learning model.

Examples include:

- Patient age.
- Previous admission count.
- Comorbidity count.
- Gender.
- Insurance information.
- BPL-card information.
- Diagnosis-related information.
- Hospital-related attributes.
- Other admission-related features.

Feature engineering helps transform raw healthcare information into variables that can be processed by the Machine Learning model.

5.6.4 Time-Based Train/Test Split

Instead of randomly dividing all records, the project uses a time-based evaluation strategy.

The dataset is divided approximately as follows:

Dataset	Period	Records
Training	2015–2023	108,097
Testing	2024	11,903

This approach is useful because the model is trained using historical information and evaluated using a later period.

5.6.5 Model Training

Multiple Machine Learning algorithms were evaluated.

Algorithm	ROC-AUC	Recall	F1-score
Random Forest	0.7585	0.6518	0.3498
Gradient Boosting	0.7639	0.0982	0.1680
XGBoost	0.7621	0.6576	0.3469
Logistic Regression	0.7668	0.6884	0.3468

Based on the project's evaluation, **Logistic Regression** was selected as the primary model. The selected model uses class balancing to address the imbalanced readmission target.

5.6.6 Model Evaluation

The final Logistic Regression model produced the following test results:

Metric	Result
Accuracy	70.28%
Precision	23.18%
Recall	68.84%
F1-score	34.68%
ROC-AUC	76.68%

The confusion matrix was:

	Predicted	
	0	1
Actual 0	7427	3112
Actual 1	425	939

The model's relatively high recall indicates that it identifies a substantial proportion of actual readmission cases, which is important in a risk-screening context.

5.6.7 Model Storage

After training, the Machine Learning artifacts are stored for later use by the prediction service.

The preprocessing pipeline is saved using **Joblib**.

Training



Preprocessor



ML Model



Save Artifacts



Prediction Service

This prevents the model from needing to be retrained every time a prediction is requested.

5.7 Prediction Service Implementation

The prediction service connects the FastAPI backend with the trained Machine Learning model.

5.7.1 Probability Prediction

The Logistic Regression model generates a probability for the target outcome:

Probability of 30-Day Readmission

The result is represented between:

0% and 100%

For example:

Predicted Probability = 68.40%

This means the model estimates a relatively high probability of readmission for the supplied input.

The live prediction process is:



The service loads the trained model and preprocessing pipeline and applies them to new patient information.

5.7.2 Risk Classification

The probability is converted into one of three risk categories.

Probability Risk Level

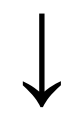
< 35% Low

35% – < 55% Medium

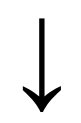
≥ 55% High

Example:

Probability = 68.40%



Risk Classification



HIGH RISK

5.7.3 Risk Factors and Clinical Information

The prediction response can provide additional information to make the result more meaningful.

The response may contain:

- Department.
- Diagnosis type.
- ICD-10 code.
- Risk factors.
- Disease-related risk factors.
- Clinical summary.
- Recommendations.

These details provide contextual information alongside the numerical prediction.

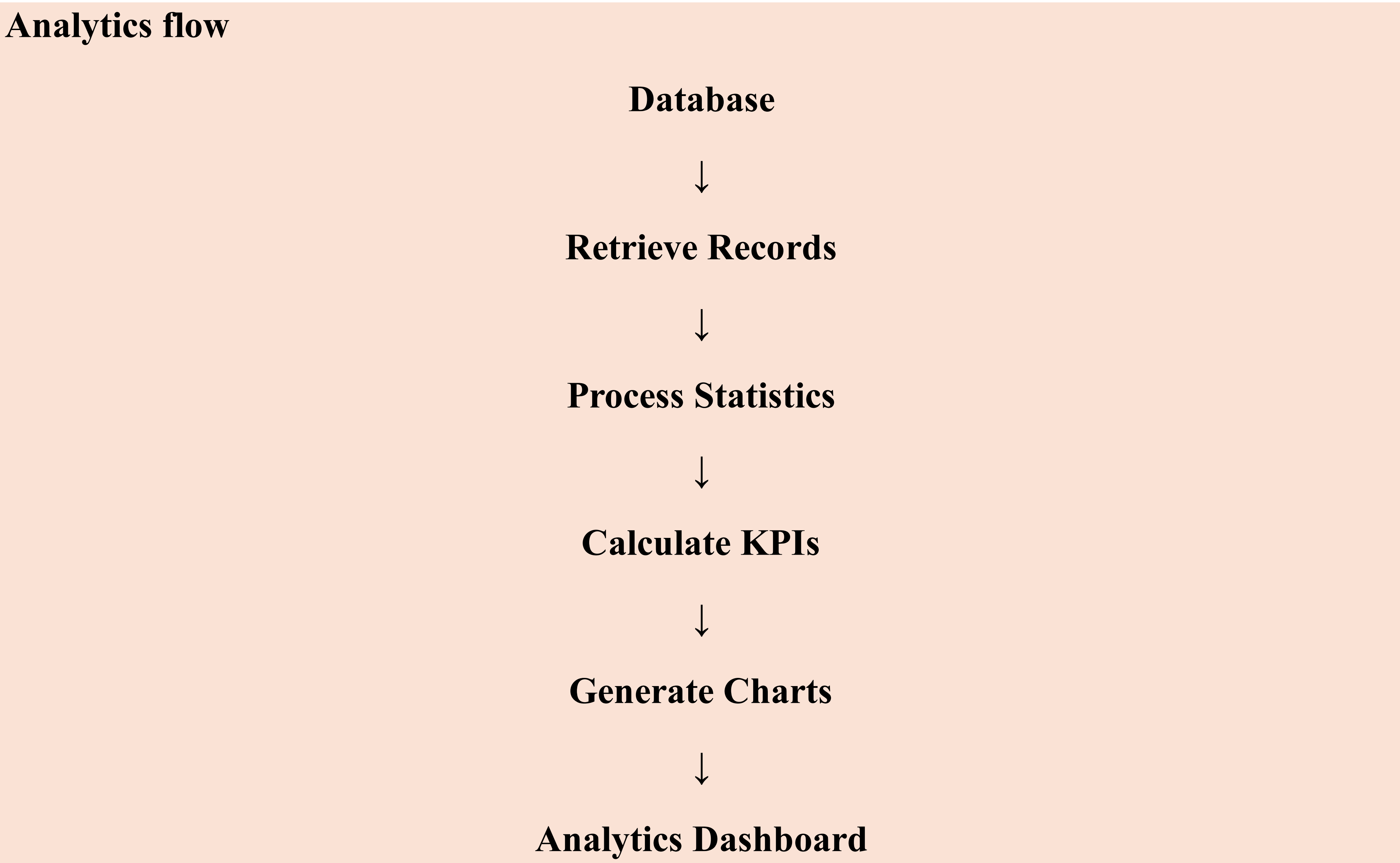
5.8 Analytics Implementation

The analytics module presents system information in a visual and understandable form.

The analytics dashboard can provide:

- Total patient count.
- Total admission count.
- Number of predictions.
- Low-risk distribution.
- Medium-risk distribution.
- High-risk distribution.
- Prediction trends.

- Model performance information.



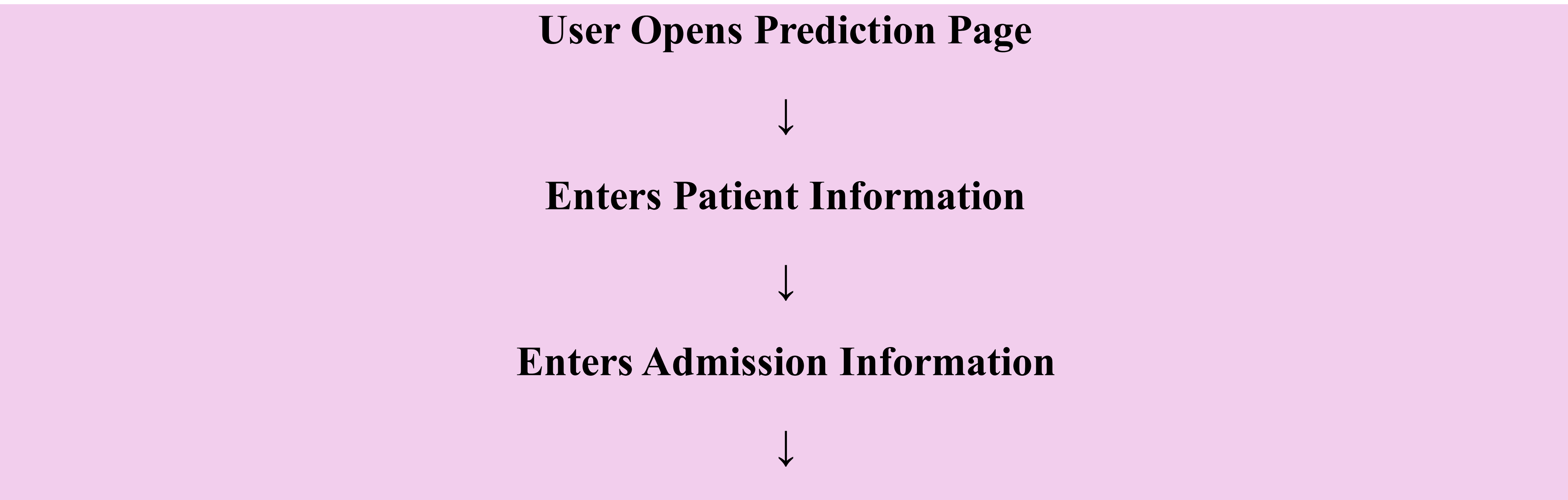
Analytics help users understand overall prediction patterns rather than viewing individual predictions only.

5.9 Live Prediction Implementation

One of the important features of the project is **live prediction**.

The user does not need to retrain the Machine Learning model for each new patient. Instead, the already trained model and preprocessing pipeline are loaded by the prediction service.

Live prediction workflow





This provides the required real-time prediction functionality of the project.

5.10 Error Handling

The application should handle invalid input and system errors appropriately.

Examples include:

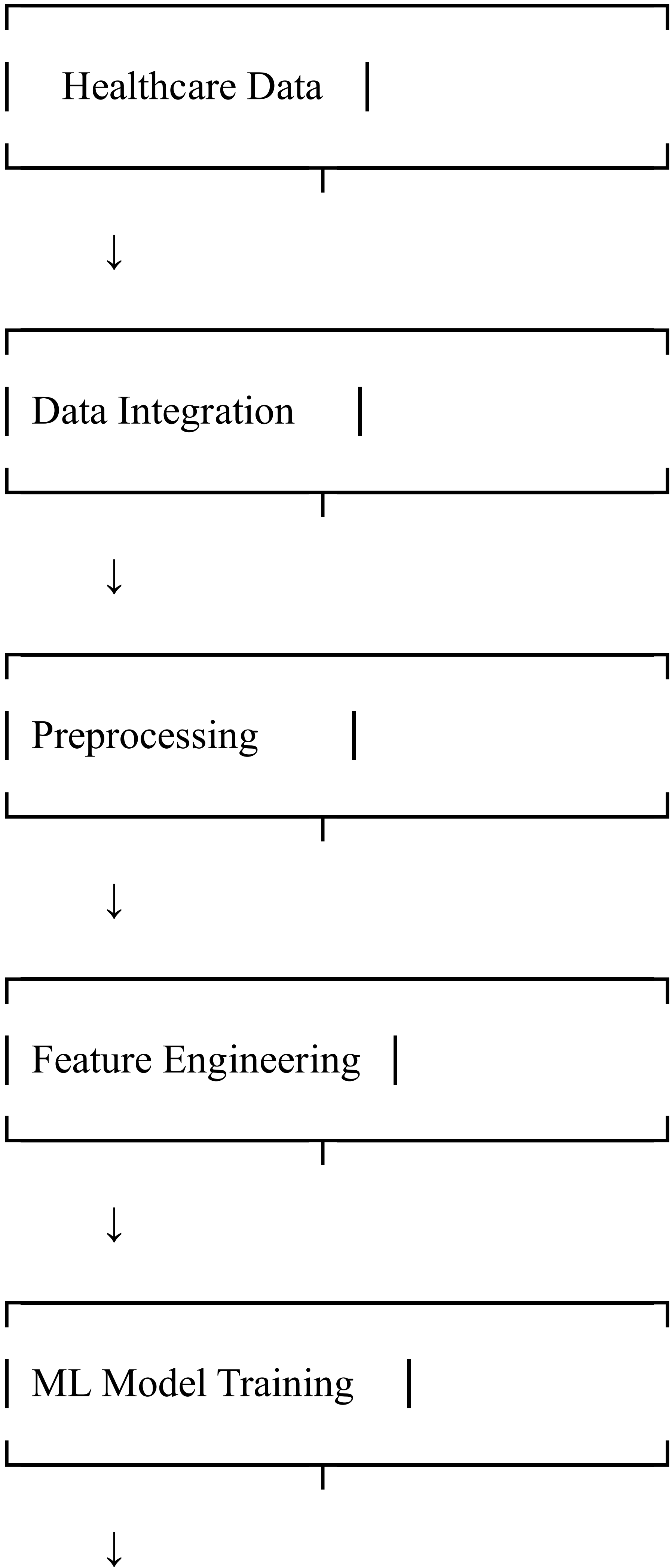
- Missing required fields.
- Invalid data types.
- Invalid authentication credentials.

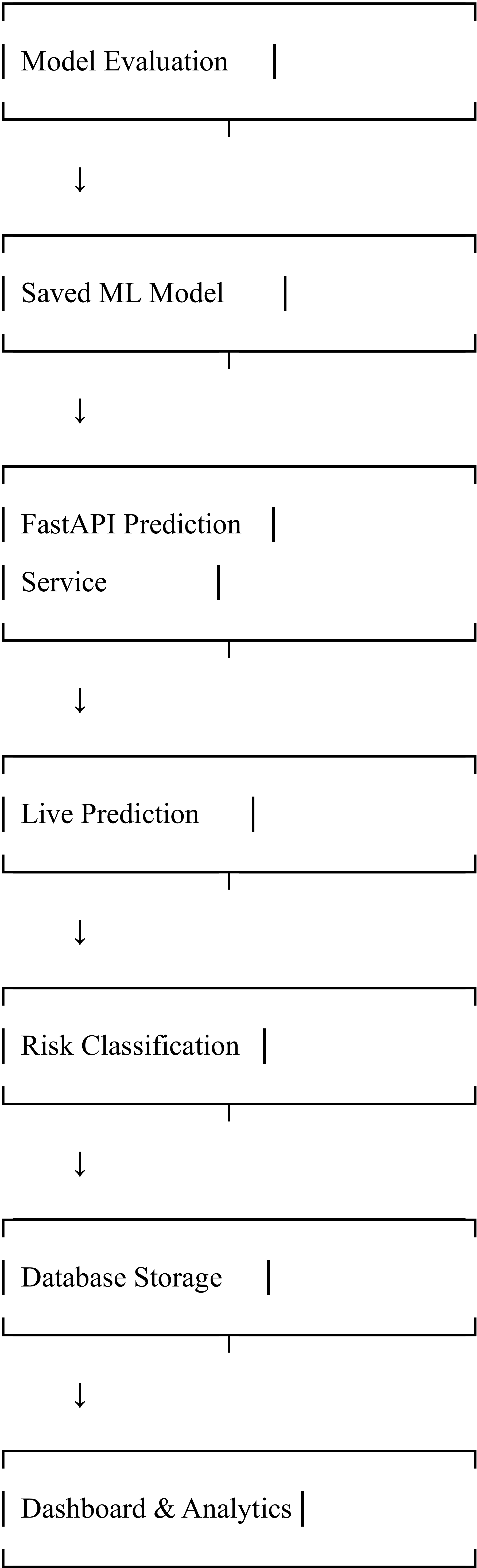
- Invalid patient or admission ID.
- Database connection errors.
- Model loading errors.
- Invalid prediction requests.

The backend returns appropriate API responses so that the frontend can display understandable messages to users.

5.11 Overall Implementation Flow

The complete implementation can be summarized as:





5.12 Chapter Summary

This chapter described the implementation of the **AI-Based Hospital Readmission Prediction System**. The system integrates a React frontend, FastAPI backend, relational database, authentication, Machine Learning model, live prediction service, and analytics dashboard.

The Machine Learning pipeline integrates healthcare data, performs preprocessing and feature engineering, evaluates multiple algorithms, and uses **Logistic Regression** as the final prediction model. The trained model is integrated with FastAPI to provide live predictions for new patient information.

The implemented system therefore provides an end-to-end workflow:

Healthcare Data → Machine Learning → FastAPI → Live Prediction → Risk Classification → Database → React Dashboard → Analytics

CHAPTER 6

TESTING

6.1 Introduction

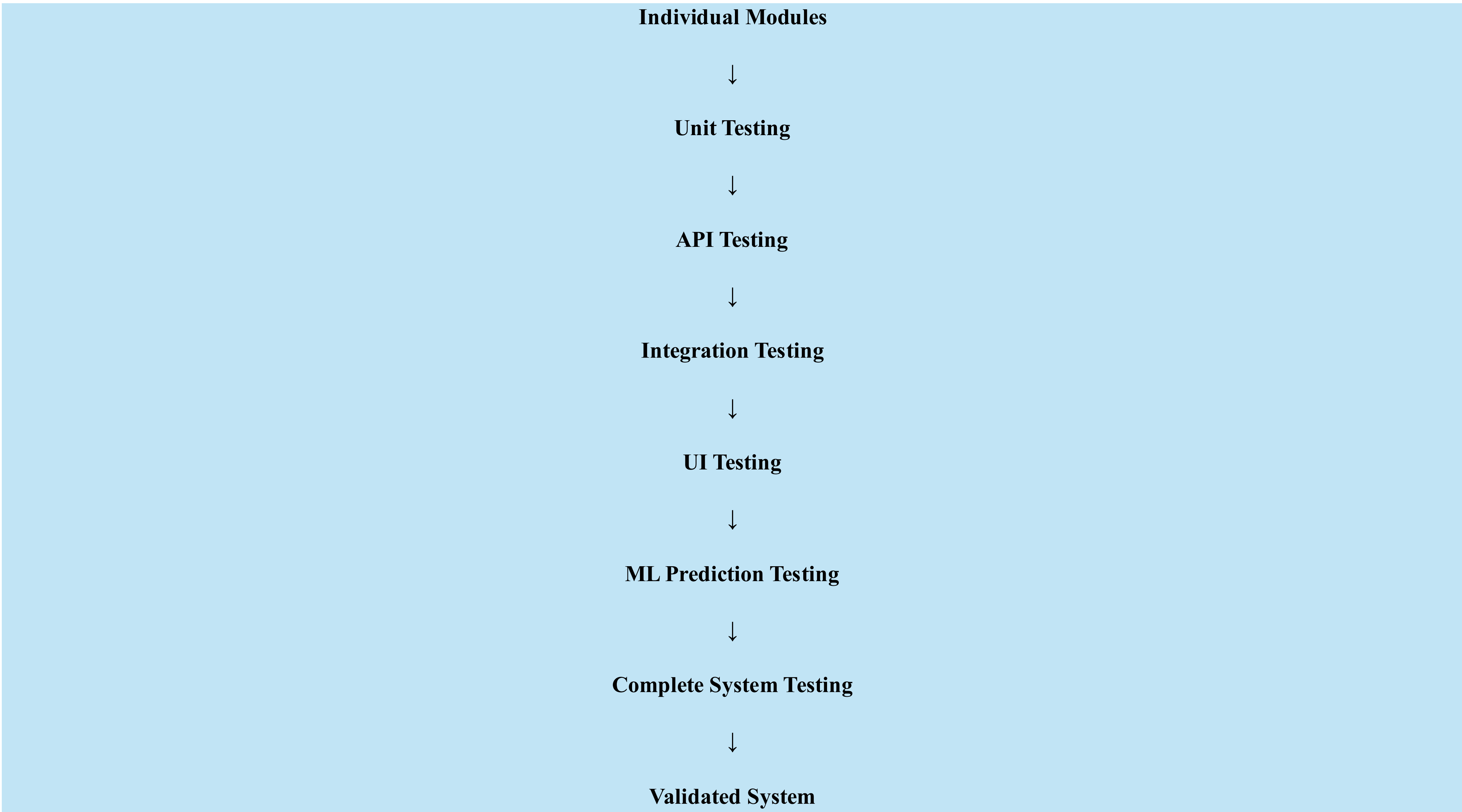
Testing is an essential phase in the development of the **AI-Based Hospital Readmission Prediction System**. The main purpose of testing is to verify that all components of the application work correctly, satisfy the specified requirements, and interact with each other as expected.

The system contains several interconnected components, including the **React frontend, authentication module, FastAPI backend, database, Machine Learning model, live prediction service, and analytics dashboard**. Therefore, testing is performed at different levels to identify functional errors, integration issues, invalid inputs, and unexpected system behavior.

The major testing activities performed for the system are:

- Unit Testing
- API Testing
- Integration Testing
- UI Testing
- Machine Learning Model Testing
- Test Case Validation

The overall testing process can be represented as:



6.2 Objectives of Testing

The main objectives of testing the proposed system are:

1. To verify that each individual module works correctly.
 2. To verify that APIs accept valid requests and return appropriate responses.
 3. To ensure that authentication prevents unauthorized access.
 4. To verify correct communication between frontend and backend.
 5. To verify correct database operations.
 6. To ensure that live prediction produces valid results.
 7. To verify correct risk classification.
 8. To ensure that prediction records are stored correctly.
 9. To verify that analytics display appropriate information.
 10. To identify and handle invalid inputs and system errors.
-

6.3 Unit Testing

Unit testing focuses on testing individual components independently. Each module is tested to determine whether it performs its intended operation correctly.

Modules considered for unit testing

- Authentication functions
 - Data preprocessing functions
 - Feature processing
 - Prediction service
 - Risk classification
 - Database operations
 - API validation
 - Analytics calculations
-

6.3.1 Authentication Unit Testing

The authentication module is tested with different credential combinations.

Examples

- Valid username and password.
- Invalid username.
- Invalid password.
- Empty credentials.
- Inactive user account.

The expected result is that only valid and authorized credentials allow access to protected application features.

6.3.2 Data Preprocessing Unit Testing

The preprocessing component is tested to verify that healthcare data is correctly transformed.

Testing includes:

- Missing numerical values.
- Missing categorical values.
- Numerical scaling.
- Categorical encoding.
- Unknown categorical values.
- Incorrect input types.

The preprocessing pipeline must produce data in the format expected by the trained Machine Learning model.

6.3.3 Prediction Service Unit Testing

The prediction service is tested independently to verify that it:

1. Receives valid input.
2. Loads the saved preprocessing pipeline.
3. Loads the trained ML model.
4. Generates a probability.
5. Classifies the probability into a risk category.
6. Returns the expected prediction response.

6.3.4 Risk Classification Unit Testing

The risk-classification logic is tested against the defined thresholds.

Probability Expected Result

20%	Low Risk
34.9%	Low Risk
35%	Medium Risk
45%	Medium Risk
54.9%	Medium Risk
55%	High Risk
80%	High Risk

This verifies that boundary conditions are handled correctly.

6.4 API Testing

API testing verifies communication between the frontend and FastAPI backend.

The APIs are tested using valid and invalid requests.

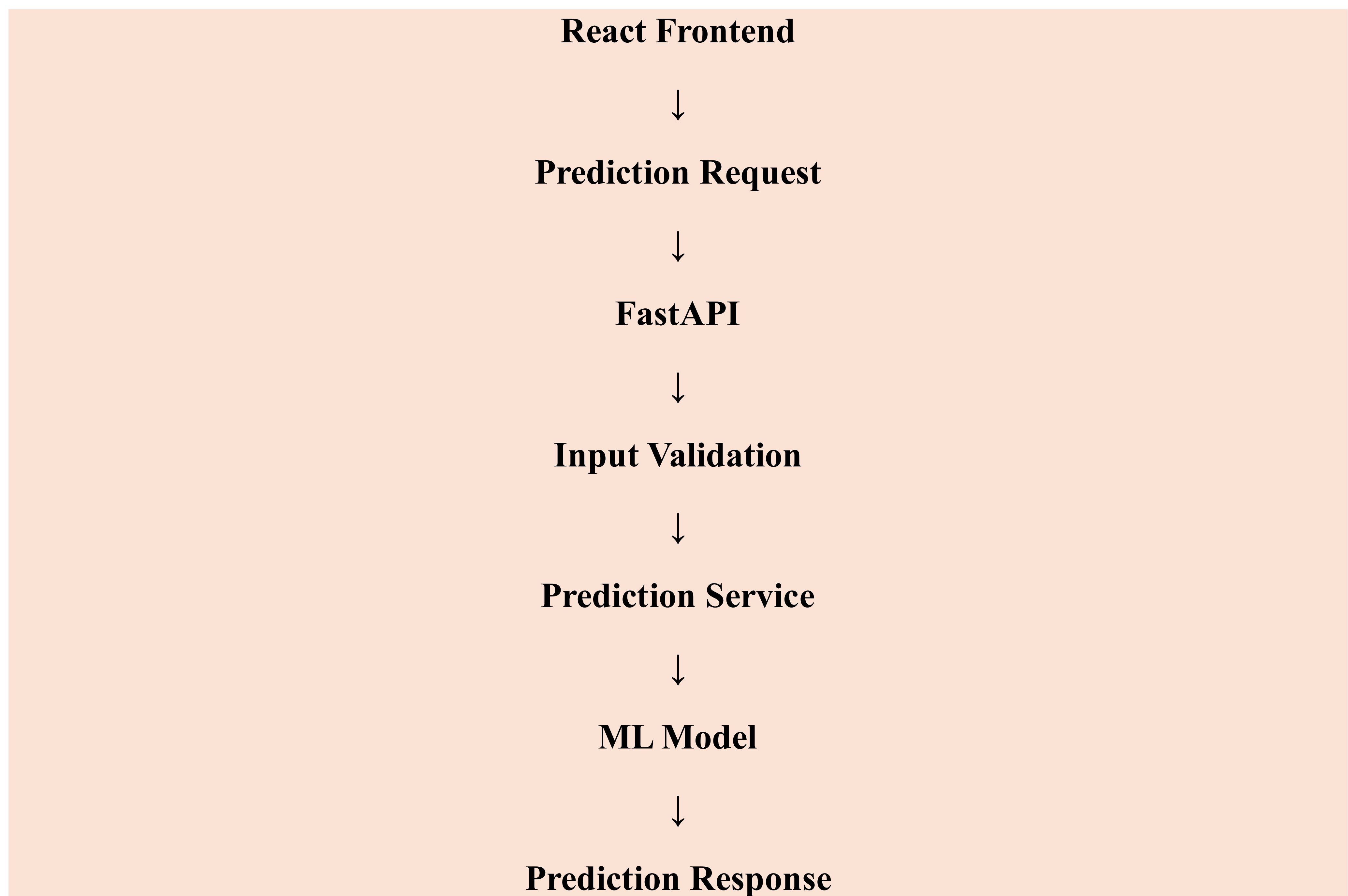
Main API operations

- Authentication API
- Patient API
- Admission API
- Prediction API
- Prediction history API
- Analytics API
- Health/status API

6.4.1 Prediction API Testing

The prediction API is one of the most important APIs in the application.

Request flow



The API is tested for:

- Valid patient information.
- Missing fields.
- Invalid values.
- Incorrect data types.
- Prediction generation.
- Response structure.
- Risk classification.

6.4.2 API Response Validation

The prediction response is checked to ensure that the expected fields are returned.

Example response structure:

```
{  
  admission_id,  
  probability,  
  probability_percent,  
}
```



```
risk_level,  
department_name,  
diagnosis_type,  
risk_factors,  
clinical_summary,  
recommendations,  
model_name,  
model_version  
}
```

The response should contain valid values and an appropriate HTTP response status.

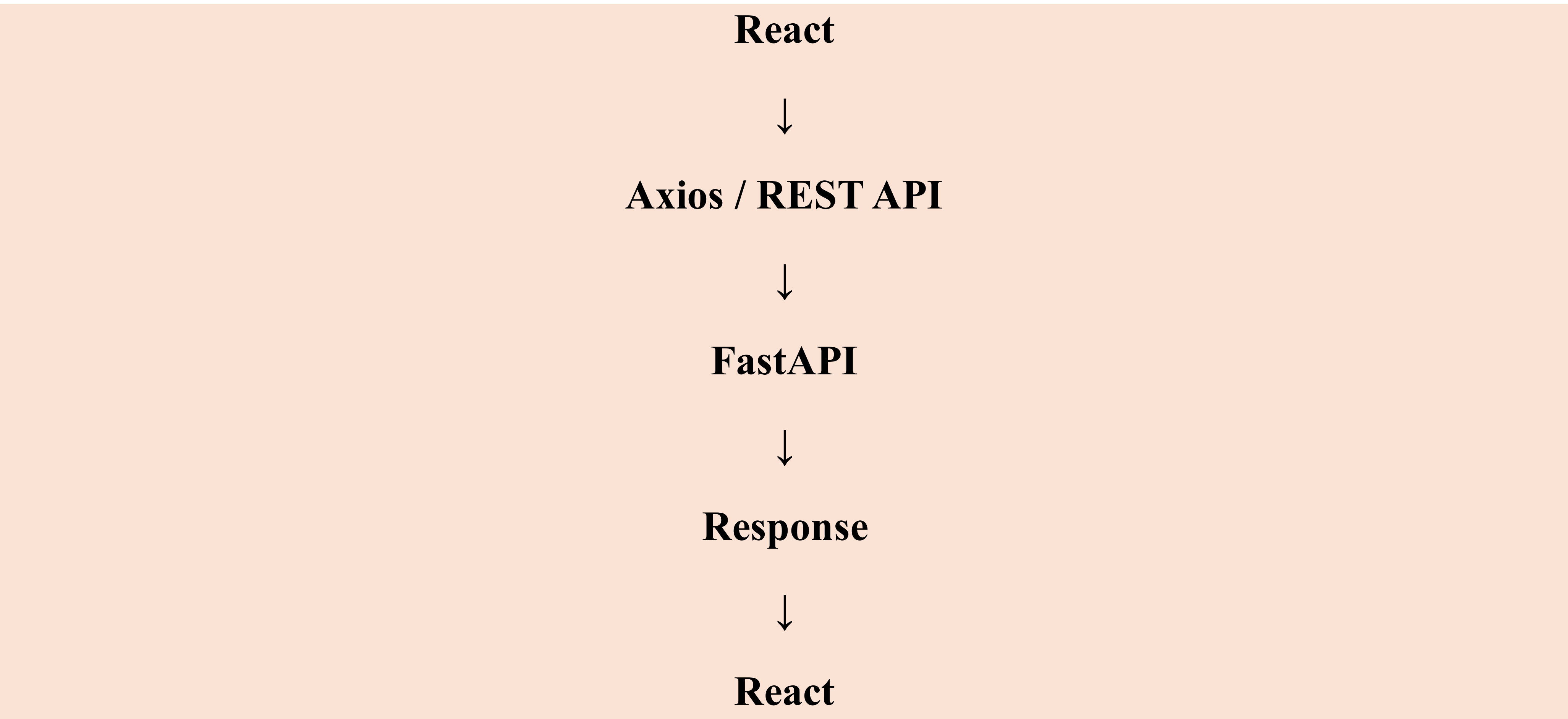
6.5 Integration Testing

Integration testing verifies whether different modules work correctly when connected.

The proposed system contains several important integrations.

6.5.1 Frontend–Backend Integration

The React frontend sends requests to the FastAPI backend.



Testing verifies that:

- Requests are correctly sent.
- Data is correctly received.
- Errors are appropriately handled.

- Results are displayed correctly.

6.5.2 Backend–Database Integration

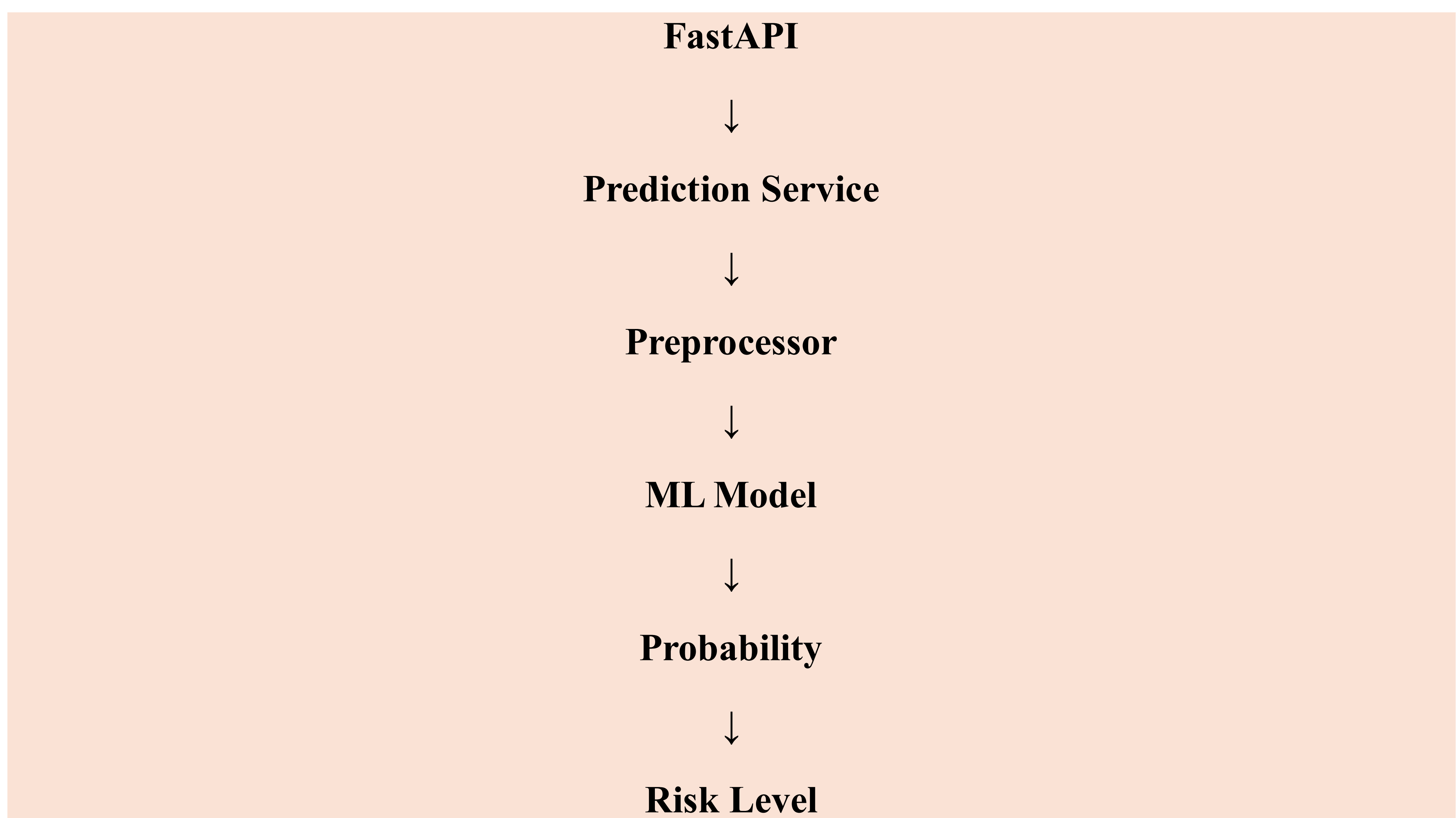
The FastAPI backend communicates with the relational database through SQLAlchemy.

Testing verifies:

- User records can be retrieved.
- Patient records can be created and retrieved.
- Admissions can be associated with patients.
- Predictions can be stored.
- Historical predictions can be retrieved.

6.5.3 Backend–ML Integration

The prediction service connects the FastAPI backend to the trained Machine Learning model.



Testing verifies that the saved model and preprocessing pipeline can be loaded successfully and used for live prediction.

6.5.4 Prediction–Database Integration

After a prediction is generated, the prediction result is stored in the database.

Prediction Generated



Prediction Record



Database



Prediction History

This ensures that prediction results remain available for later review.

6.6 UI Testing

UI testing verifies whether the React frontend behaves correctly from the user's perspective.

The following interfaces are tested:

6.6.1 Login Page

Testing includes:

- Correct login credentials.
- Incorrect credentials.
- Empty fields.
- Login button.
- Error messages.
- Navigation after successful login.

6.6.2 Dashboard

Testing verifies:

- Dashboard loads correctly.
- Statistics are displayed.
- Navigation options are available.
- Analytics information is loaded correctly.

6.6.3 Patient Management

Testing includes:

- Patient creation.
 - Patient viewing.
 - Patient search.
 - Patient information display.
 - Form validation.
-

6.6.4 Admission Management

Testing includes:

- Admission creation.
 - Patient association.
 - Admission information display.
 - Admission history.
-

6.6.5 Live Prediction Page

The live prediction page is tested for:

- Input form loading.
 - Input validation.
 - Prediction submission.
 - Loading state.
 - Prediction response.
 - Probability display.
 - Risk-level display.
 - Error handling.
-

6.6.6 Prediction History

Testing verifies:

- Previous predictions are displayed.
- Prediction probability is shown.

- Risk level is displayed.
- Associated admission information is shown.
- Records are retrieved correctly.

6.6.7 Analytics Dashboard

Testing verifies:

- Statistics are displayed.
- Risk distribution is represented correctly.
- Charts load correctly.
- Data corresponds to stored records.

6.7 Machine Learning Model Testing

Machine Learning testing verifies whether the trained model performs correctly on unseen data.

The project uses a **time-based test dataset** representing a later period than the training data.

Test dataset

- Training period: **2015–2023**
- Testing period: **2024**
- Training records: **108,097**
- Testing records: **11,903**

The final Logistic Regression model produced:

Metric	Result
Accuracy	70.28%
Precision	23.18%
Recall	68.84%
F1-score	34.68%
ROC-AUC	76.68%

The confusion matrix was:

	Predicted	
	No	Yes
Actual No	7427	3112
Actual Yes	425	939

The model testing confirms that the trained model can generate predictions on the held-out test period.

6.8 Live Prediction Testing

Live prediction testing verifies the complete prediction workflow using newly entered patient information.

Testing flow



The following conditions are tested:

- Valid patient information.
- Missing required information.
- Invalid numerical values.
- Valid categorical values.
- Prediction generation.
- Probability range.
- Risk classification.
- Database storage.
- Frontend result display.

6.9 Test Cases

The following table presents representative test cases for the system.

Test ID	Module	Test Condition	Expected Result	Status
TC-01	Login	Valid credentials	User successfully logged in	Pass
TC-02	Login	Invalid password	Error message displayed	Pass
TC-03	Login	Empty fields	Validation message displayed	Pass
TC-04	Patient	Add valid patient	Patient record created	Pass
TC-05	Patient	View patient	Patient details displayed	Pass
TC-06	Admission	Create valid admission	Admission created	Pass
TC-07	Admission	Invalid patient reference	Appropriate error returned	Pass
TC-08	Prediction	Valid prediction input	Prediction generated	Pass
TC-09	Prediction	Missing required input	Validation error returned	Pass
TC-10	Prediction	Invalid input type	Request rejected	Pass

TC-11	Risk Classification	Probability below 35%	Low Risk	Pass
TC-12	Risk Classification	Probability 35–54.99%	Medium Risk	Pass
TC-13	Risk Classification	Probability $\geq$ 55%	High Risk	Pass
TC-14	Database	Save prediction	Prediction stored	Pass
TC-15	History	Retrieve predictions	Historical records displayed	Pass
TC-16	Analytics	Load dashboard	Statistics displayed	Pass
TC-17	API	Valid API request	Successful response	Pass
TC-18	API	Invalid API request	Appropriate error response	Pass
TC-19	ML Model	Test unseen data	Prediction generated	Pass
TC-20	Integration	Frontend-to-backend prediction	Result displayed correctly	Pass

6.10 Boundary Value Testing

Boundary testing is particularly important for the risk-classification module because the system uses defined probability thresholds.

Test Input Expected Risk

0%	Low
34.99%	Low
35%	Medium
35.01%	Medium
54.99%	Medium
55%	High
55.01%	High

Test Input Expected Risk

100% High

This ensures that the classification logic behaves correctly at threshold boundaries.

6.11 Validation Testing

Validation testing verifies that the system correctly handles invalid information.

Examples

Invalid Input	Expected Behavior
Empty required field	Display validation error
Negative age	Reject invalid value
Invalid categorical value	Reject or validate input
Invalid patient ID	Return appropriate error
Invalid admission ID	Return appropriate error
Incorrect login password	Deny authentication
Missing prediction fields	Reject prediction request
Database unavailable	Return controlled error

Proper validation prevents incorrect data from entering the prediction pipeline.

6.12 Security Testing

Security testing verifies that application resources are protected.

The following areas are considered:

- Login authentication.
- Password protection.
- Protected API endpoints.
- Role-based access.
- Unauthorized access attempts.
- Input validation.

- Database access control.

An unauthenticated user should not be able to access protected application modules.

6.13 Performance Testing

Performance testing evaluates how efficiently the application responds to user requests.

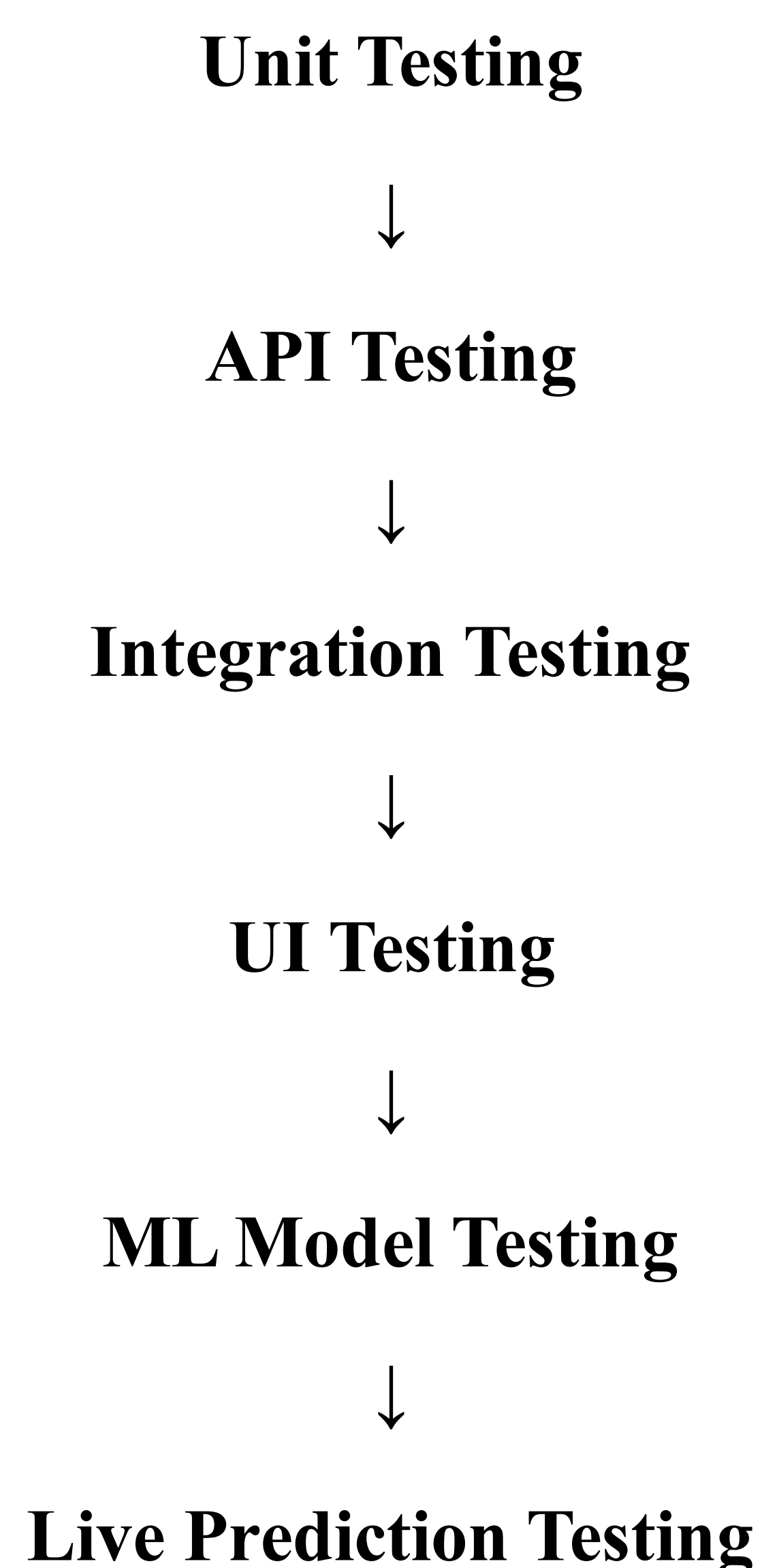
The following areas can be evaluated:

- API response time.
- Prediction response time.
- Database query performance.
- Dashboard loading time.
- Frontend rendering.
- Model loading behavior.

The trained model is saved and reused for predictions, avoiding unnecessary model retraining for every live prediction request.

6.14 Testing Summary

The testing process covers the complete application from individual functions to end-to-end system behavior.



↓
Security Testing

↓
Performance Testing

↓
Final System Validation

The testing process confirms that the major components of the system are designed to work together:

Authentication → React Frontend → FastAPI Backend → Prediction Service → ML Model → Database → Analytics

6.15 Chapter Summary

This chapter presented the testing methodology for the AI-Based Hospital Readmission Prediction System. Unit testing was used to verify individual modules, while API testing evaluated backend services. Integration testing verified communication between the frontend, backend, database, and Machine Learning components.

UI testing covered the major application screens, including login, dashboard, patient management, admission management, live prediction, prediction history, and analytics. Machine Learning testing evaluated the selected Logistic Regression model using a time-based test dataset.

The testing process also included validation, boundary-value testing, security considerations, and live prediction testing. These activities help ensure that the system operates reliably and that valid patient information can flow through the application to produce an appropriate 30-day readmission probability and risk classification.

CHAPTER 7

RESULTS AND DISCUSSION

Introduction

The **AI-Based Hospital Readmission Prediction System** was developed as an integrated healthcare application that combines Machine Learning with a web-based hospital management interface. The system processes historical healthcare data, trains a Machine Learning model, generates readmission probabilities, classifies patients into different risk levels, and provides live prediction through the application.

The results of the project are evaluated from two perspectives:

1. **Machine Learning results** – performance of the trained prediction models.
2. **Application results** – functionality of the implemented web application, including login, dashboard, patient management, prediction, prediction history, and analytics.

The major system workflow is:

Healthcare Data → Data Integration → Preprocessing → Feature Engineering → ML Model → Risk Probability → Risk Classification → Live Prediction → Database → Dashboard & Analytics

7.1 Login and Authentication

The system provides an authentication mechanism before allowing users to access protected healthcare functionality.

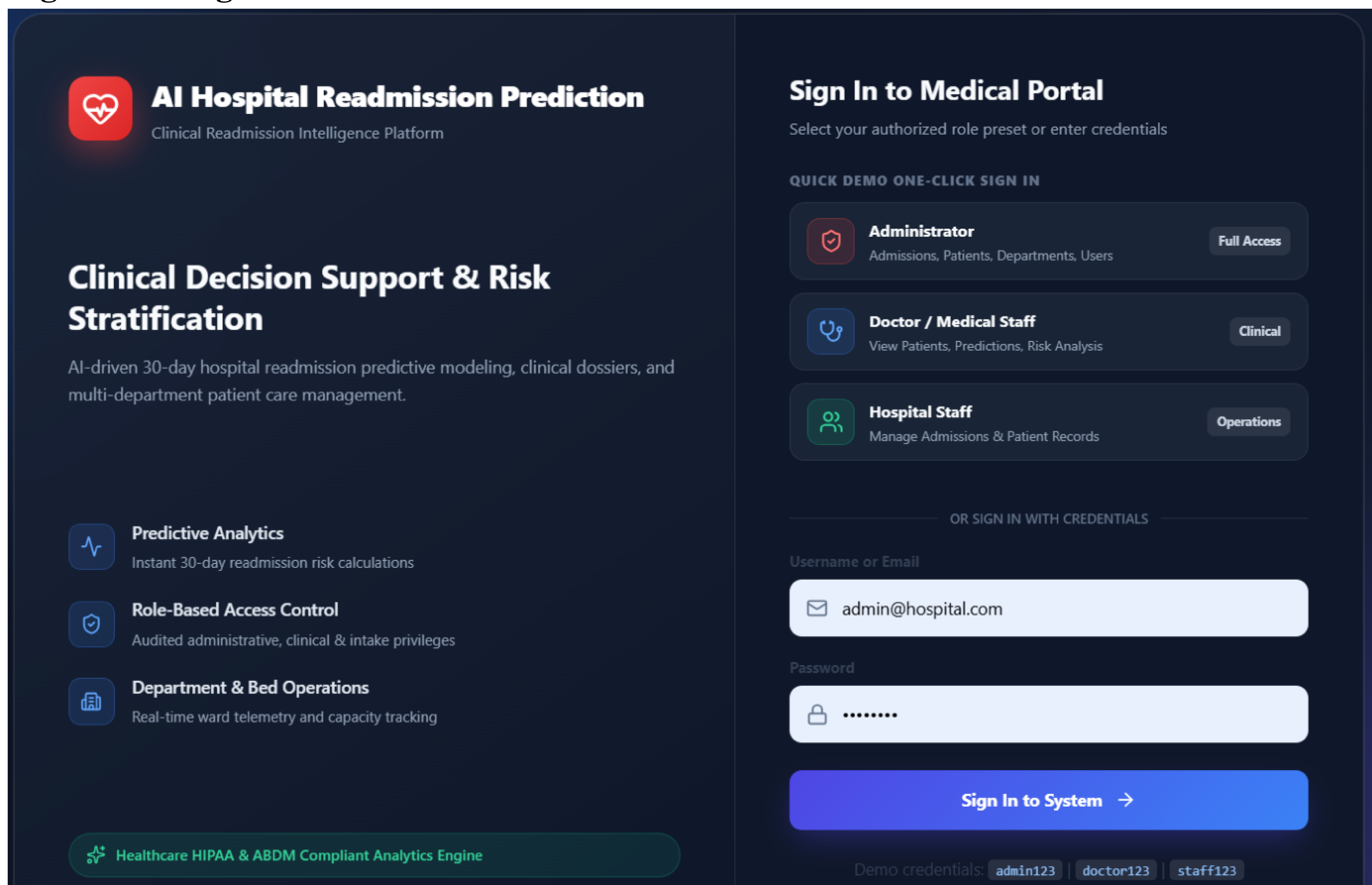
The login interface allows an authorized user to enter:

- Username/email
- Password

After successful authentication, the user is redirected to the main dashboard. Invalid credentials result in an appropriate error message.

Authentication helps prevent unauthorized access to patient and prediction information.

Figure 7.1 Login Screen



The login module provides the initial security layer of the application and ensures that healthcare-related information is accessible only to authorized users.

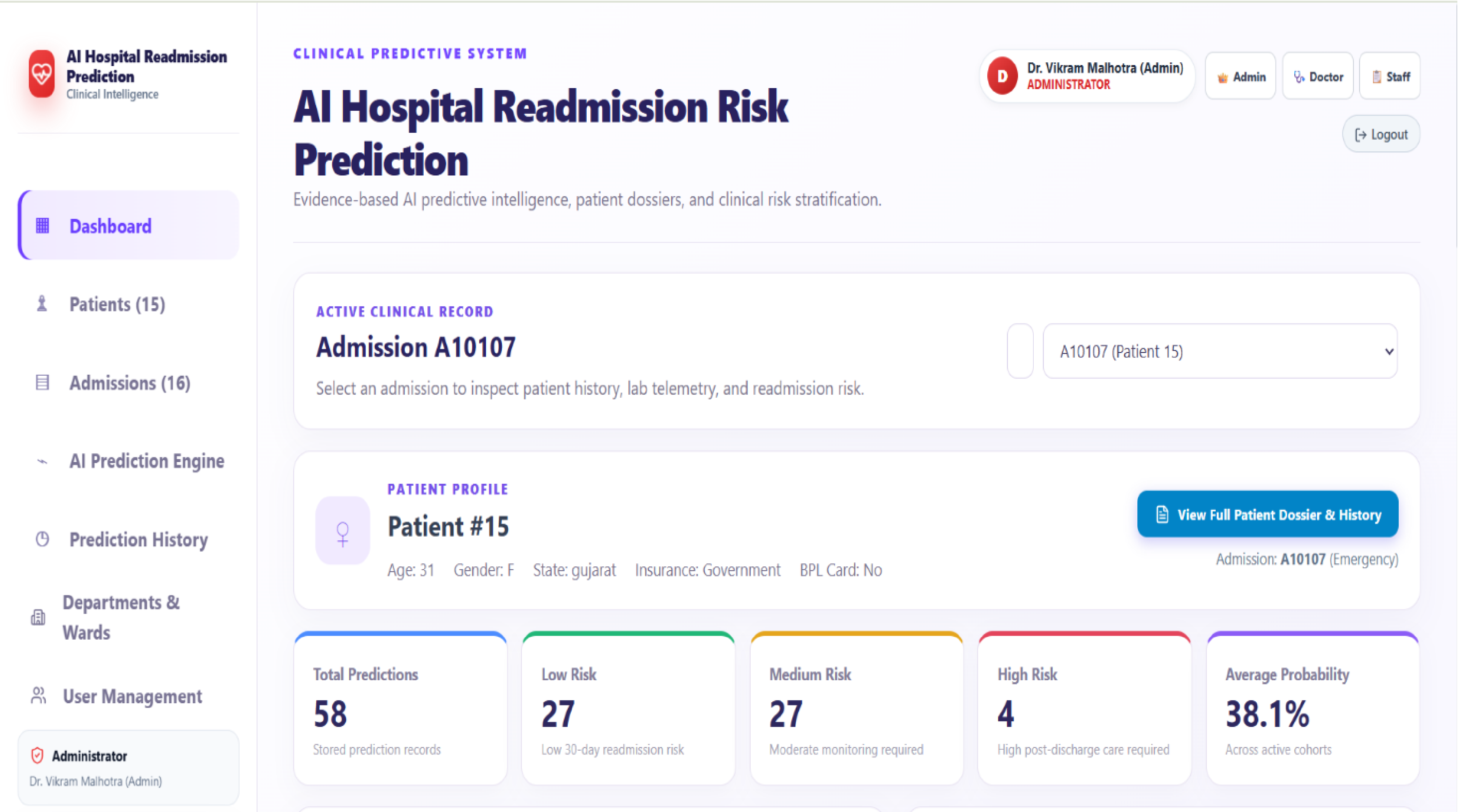
7.2 Dashboard

After successful login, the user is presented with the main dashboard.

The dashboard provides an overview of the system and can display important information such as:

- Total patients
- Total admissions
- Total predictions
- High-risk patients
- Medium-risk patients
- Low-risk patients
- Recent prediction activity

Figure 7.2 Dashboard



The dashboard provides a centralized view of system activity. It allows users to quickly understand the current prediction workload and risk distribution without manually reviewing individual records.

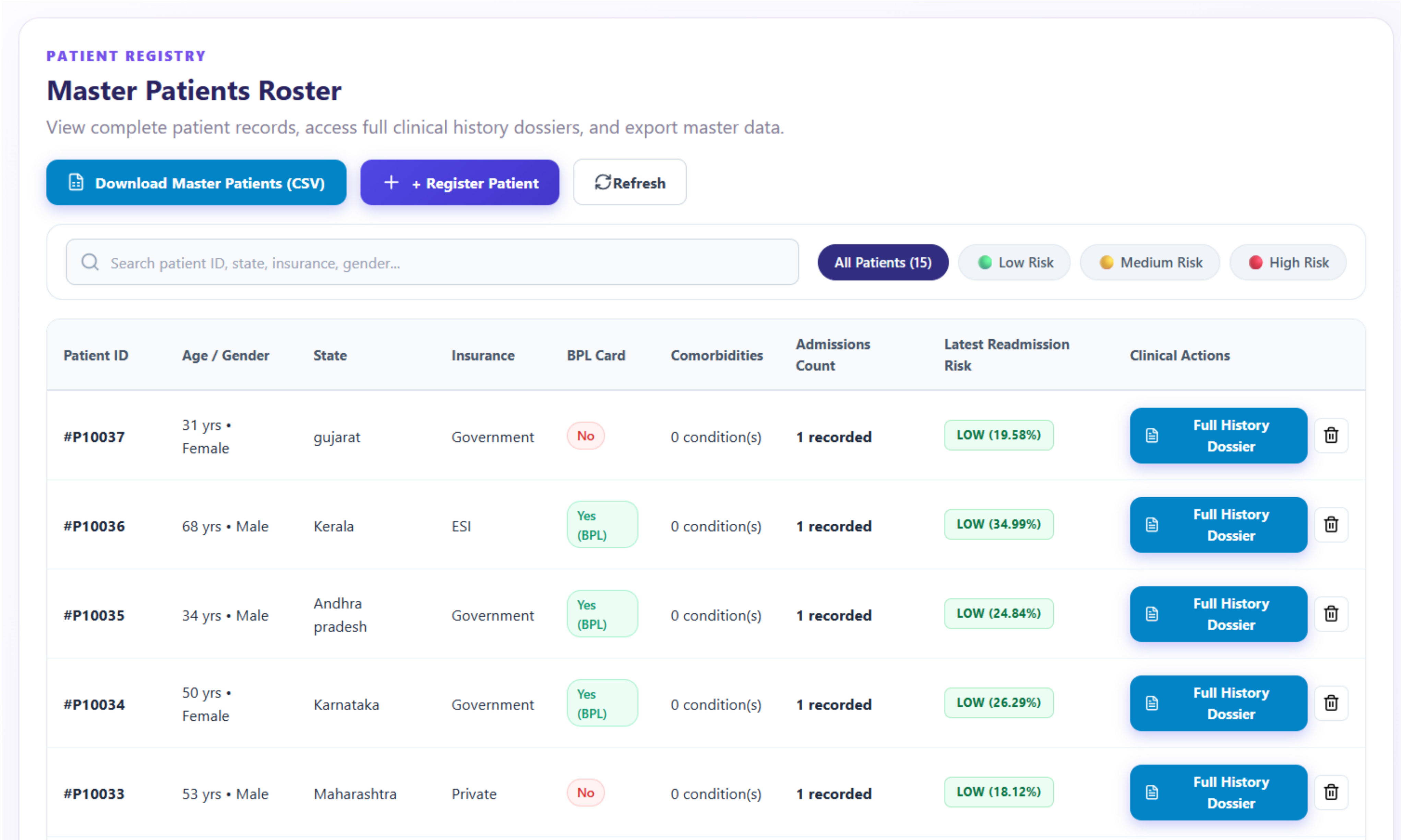
7.3 Patient Management

The Patient Management module is used to maintain patient-related information.

The system can store information such as:

- Patient ID
- Age
- Gender
- State
- Insurance type
- Comorbidity information
- Previous admissions

Figure 7.3 Patient Management Screen



The patient management functionality provides a structured way to maintain patient information. This information can subsequently be used as input to the prediction process. The module also helps maintain consistency between patient records and their associated admissions and predictions.

7.4 Live Readmission Prediction

One of the major features of the project is **live prediction**. The user can enter the required patient and admission information through the prediction interface. The frontend sends the information to the FastAPI backend through a REST API.

The prediction process follows:

Patient Input → API Request → Data Validation → Preprocessing → Logistic Regression → Probability → Risk Classification → Result

The model generates a probability between 0 and 1.

For example, if the model produces:

Probability = 0.72

the system converts this value into:

Risk Level = HIGH

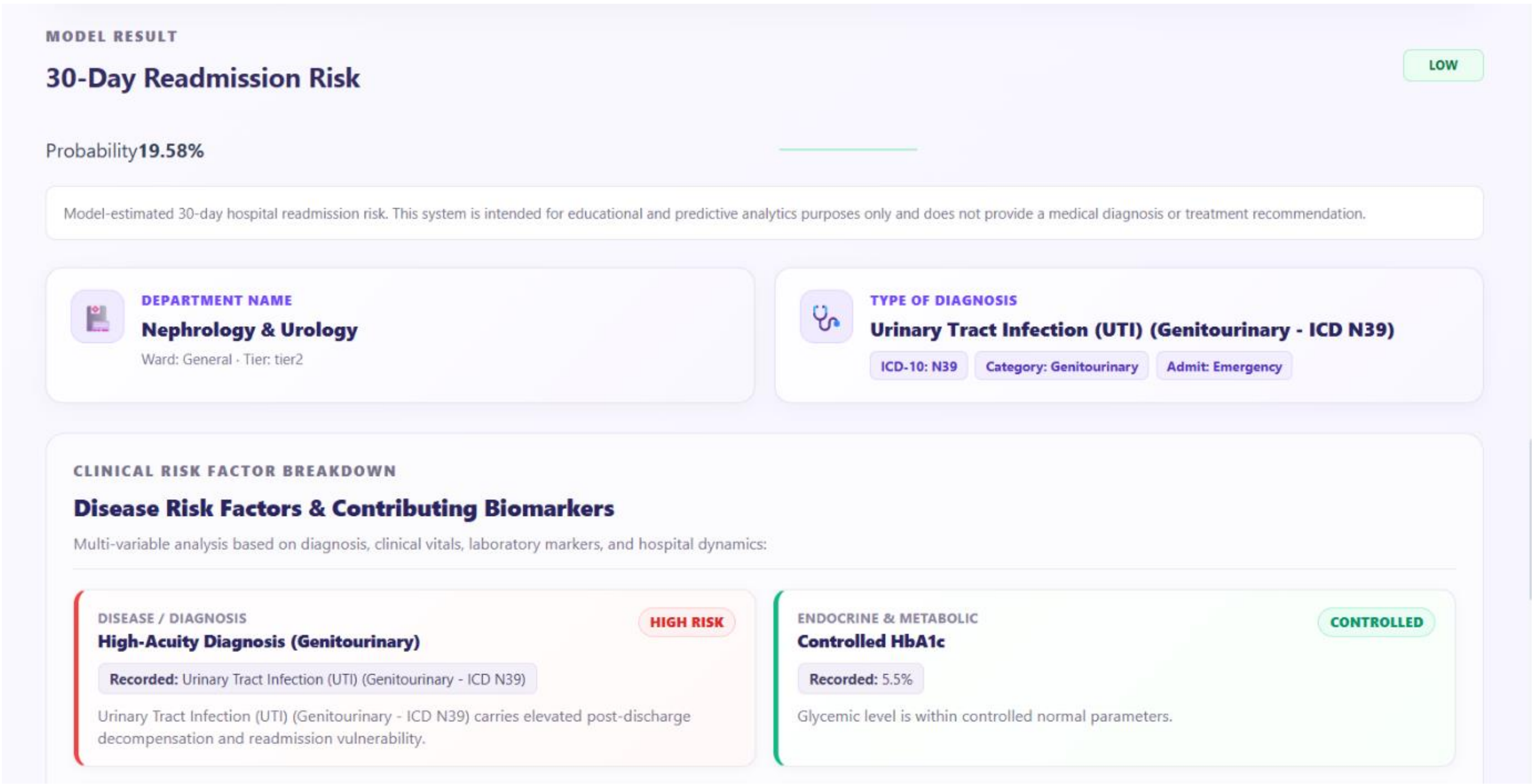
7.41Prediction Result

After processing the submitted patient information, the system displays the prediction result.

The result can contain:

- Admission ID
- Readmission probability
- Probability percentage
- Risk level
- Department name
- Diagnosis type
- ICD-10 code
- Risk factors
- Disease-related risk factors
- Clinical summary
- Recommendations
- Model name
- Model version

Figure 7.41 Prediction Result



The probability-based output provides more information than a simple binary classification. A healthcare professional can identify whether the patient's predicted risk is low, moderate, or high.

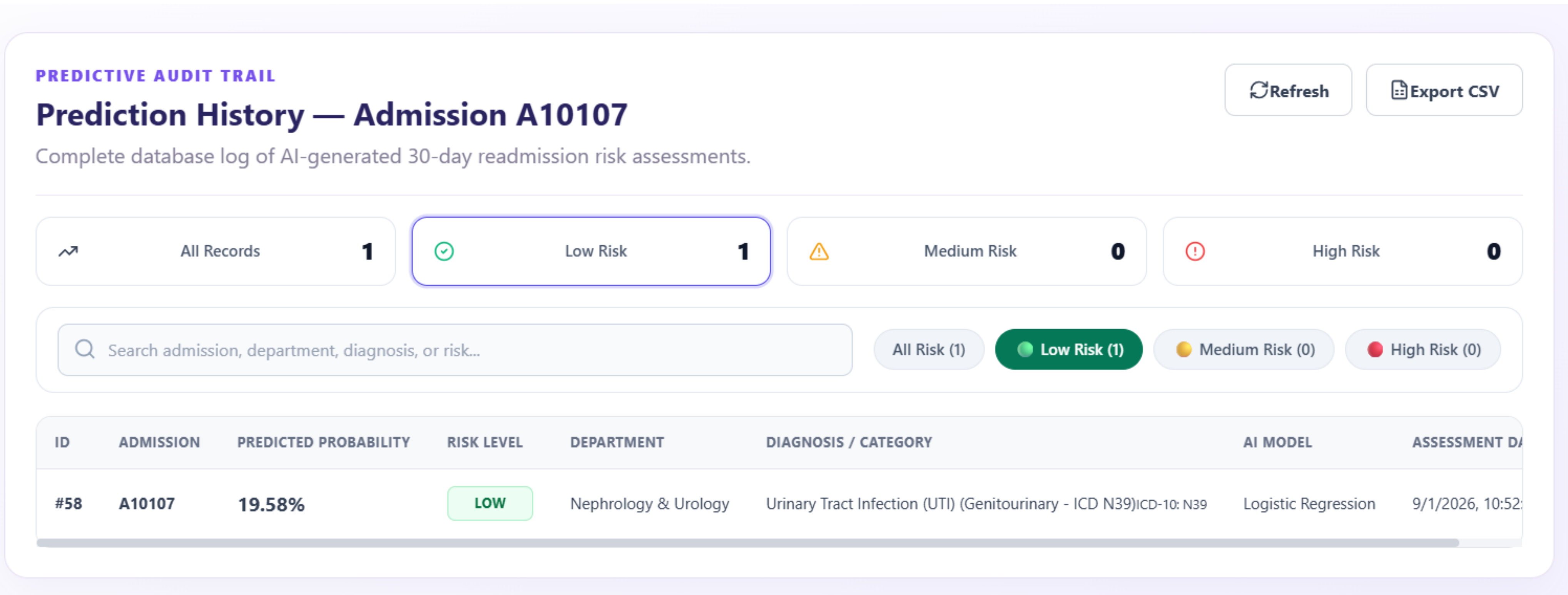
The result is intended to support healthcare decision-making and should not be considered a replacement for professional medical judgment.

7.5Prediction History

The Prediction History module stores previously generated prediction results. It allows users to review previous predictions without generating the same prediction again. The history can include:

- Patient/admission ID
- Prediction probability
- Risk level
- Diagnosis
- Department
- Prediction data

Figure 7.6 Prediction History



- This feature is useful for tracking prediction activity and reviewing previously assessed patients.
- It also provides a historical record that can support system monitoring and analysis.

7.6 Analytics Dashboard

- The Analytics Dashboard provides a graphical representation of prediction-related information.

- Possible analytics include:
- Total predictions
- Low-risk patients
- Medium-risk patients
- High-risk patients
- Readmission distribution
- Department-level statistics
- Diagnosis-level information
- Prediction trends
- Visual analytics make large amounts of healthcare information easier to understand. Hospital administrators and authorized users can use these statistics to identify patterns and monitor readmission-related activity.

The implemented application provides **authentication, dashboard, patient management, admission management, live readmission prediction, risk classification, prediction history, database storage, and analytics**.

Overall, the project demonstrates an end-to-end integration of **Data Processing + Machine Learning + FastAPI + Database + Authentication + React Frontend + Analytics** for hospital readmission risk prediction.

CHAPTER 8

ADVANTAGES AND LIMITATIONS

8.1 Advantages of the Proposed System

8.1.1 Machine Learning-Based Prediction

The system uses Machine Learning to analyze patient and admission-related information and estimate the probability of 30-day readmission. This provides a data-driven approach to risk assessment.

8.1.2 Probability-Based Risk Assessment

Instead of producing only a Yes/No result, the system generates a probability between **0% and 100%**. This probability is converted into three understandable risk categories:

Probability	Risk Level
Below 35%	Low
35% – Below 55%	Medium
55% or above	High

8.1.3 Live Prediction

The system supports live prediction through the web interface. Users can enter new patient and admission information and receive the predicted probability and risk level without retraining the model.

8.1.4 Multiple Healthcare Data Sources

The system integrates information from patient, admission, diagnosis, hospital, and billing datasets. Combining multiple sources provides a broader view of patient and admission characteristics.

8.1.5 Centralized Database

Patient, admission, and prediction information can be maintained in a centralized relational database. This supports efficient retrieval, prediction-history management, and analytics.

8.1.6 Authentication and Access Control

Authentication helps prevent unauthorized access to the application. Role-based access can also be used to provide users with appropriate permissions.

8.1.7 User-Friendly Interface

The React frontend provides an organized interface for accessing:

- Dashboard

- Patient Management
- Admission Management
- Live Prediction
- Prediction History
- Analytics

8.1.8 Prediction History

Generated predictions are stored in the database and can be reviewed later. This helps users track previous prediction results and associated admission information.

8.1.9 Analytics Dashboard

The analytics module provides summarized information such as patient statistics, admission statistics, prediction counts, and risk distribution.

8.1.10 Model Evaluation

Multiple Machine Learning algorithms were evaluated before selecting the primary model. The evaluated algorithms include Logistic Regression, Random Forest, Gradient Boosting, and XGBoost.

8.2 Limitations of the Proposed System

8.2.1 Dataset Dependency

Prediction quality depends on the quality and completeness of the healthcare data used for training. Missing or inaccurate information can affect the reliability of predictions.

8.2.2 Class Imbalance

The dataset contains more non-readmission cases than readmission cases.

Class	Records	Percentage
No Readmission	105,790	88.16%
Readmission	14,210	11.84%

Although class balancing was considered during model training, imbalance remains a challenge.

8.2.3 False-Positive Predictions

The final Logistic Regression model achieved **23.18% precision**. Therefore, some cases predicted as readmission may not actually result in readmission.

8.2.4 Model Performance

The selected model achieved a **76.68% ROC-AUC** and **68.84% recall**. Although these results demonstrate useful predictive capability, the model is not perfectly accurate.

8.2.5 Generalization

The model is trained using the available dataset. Its performance may differ when applied to another hospital, region, or patient population.

8.2.6 Input Quality

Live prediction depends on the information entered into the system. Incorrect or incomplete patient information can lead to unreliable results.

8.2.7 Security and Privacy

Healthcare information requires strong protection. A production deployment would require appropriate encryption, access control, secure communication, audit logging, and healthcare-data compliance measures.

8.2.8 Clinical Decision Limitation

The system should be considered a **decision-support and risk-screening tool**. Its prediction should not replace professional clinical judgment or be treated as a definitive diagnosis.

8.3 Advantages and Limitations Summary

Area	Advantages	Limitations
Prediction	ML-based risk prediction	Predictions are not always correct
Probability	Provides numerical risk probability	Depends on model performance
Live Prediction	Immediate prediction	Depends on input quality
Data	Multiple healthcare sources	Data quality affects results
Database	Centralized storage	Requires proper security
Authentication	Controlled access	Requires secure credential handling
Frontend	User-friendly interface	Depends on application connectivity
Analytics	Provides useful insights	Depends on available data

AI HOSPITAL READMISSION RISK PREDICTION

Area	Advantages	Limitations

CHAPTER 9

FUTURE ENHANCEMENTS

9.1 Introduction

The **AI-Based Hospital Readmission Prediction System** provides a machine-learning-based approach for predicting the probability of patient readmission within 30 days. The system integrates healthcare data processing, machine learning, live prediction, FastAPI, database management, authentication, React frontend, and analytics.

Several enhancements can be introduced in the future to improve the system's **accuracy, scalability, security, explainability, and usability**.

9.2 Advanced Machine Learning Models

Future versions can evaluate advanced models such as **LightGBM, CatBoost, optimized XGBoost, ensemble models, and neural networks**. These models can be compared with the existing Logistic Regression model using metrics such as accuracy, precision, recall, F1-score, ROC-AUC, and PR-AUC.

This may improve the system's ability to identify patients at higher risk of readmission.

9.3 Explainable AI

Explainable AI can be incorporated to show why a particular patient received a specific risk score. Techniques such as **SHAP** can identify important factors contributing to the prediction.

The system can display:

- Important risk factors
- Feature contribution
- Patient-specific explanations
- Predicted probability
- Model confidence

This can make the prediction easier for healthcare professionals to understand.

9.4 Real-Time Hospital Data Integration

The system can be integrated with hospital information systems and **Electronic Health Records (EHR)**. Future versions can automatically receive updated information such as patient details, diagnoses, laboratory results, admissions, and billing information.

This can reduce manual data entry and support more up-to-date predictions.

9.5 Automated Alerts and Live Monitoring

The existing live prediction functionality can be extended with automated risk alerts. When a patient's predicted risk reaches a predefined high-risk level, the system can notify authorized healthcare staff.

Possible features include:

- High-risk patient alerts
- Follow-up reminders
- Discharge planning alerts
- Risk-score change notifications

Continuous monitoring can help healthcare staff prioritize patients who require additional attention.

9.6 Cloud and Mobile Deployment

The system can be deployed on cloud platforms to support larger datasets, multiple hospitals, and multiple users. Cloud deployment can provide better scalability and availability.

A mobile application can also be developed to allow authorized healthcare professionals to access:

- Patient information
- Risk scores
- Live predictions
- Prediction history
- Notifications
- Analytics

9.7 Security and Model Monitoring

Future versions can strengthen healthcare data protection through **role-based access control, encryption, secure authentication, audit logging, and secure API communication.**

A model-monitoring system can also track prediction performance, data changes, model drift, and false-positive/false-negative rates. Periodic retraining can be performed when new validated healthcare data becomes available.

9.8 Advanced Analytics and Recommendations

The analytics dashboard can be enhanced with department-wise, diagnosis-wise, monthly, and yearly readmission trends. Interactive filtering can help hospital administrators identify important patterns.

Future versions may also provide preventive-care support such as **follow-up scheduling, medication review, patient education, and discharge planning recommendations**. These recommendations should support, rather than replace, professional clinical judgment.

9.9 Summary

Future Enhancement	Benefit
Advanced ML Models	Improved prediction capability
Explainable AI	Understandable predictions
EHR Integration	Updated patient information
Automated Alerts	Early identification of high-risk patients
Cloud Deployment	Scalability and availability
Mobile Application	Easy accessibility
Enhanced Security	Better data protection
Model Monitoring	Long-term reliability
Advanced Analytics	Better decision support
Preventive Recommendations	Improved readmission management

The proposed system provides a strong foundation for AI-based

CHAPTER 10

CONCLUSION

10.1 Introduction

The **AI-Based Hospital Readmission Prediction System** was developed to predict the probability of a patient being readmitted to the hospital within 30 days. The system combines healthcare data processing, machine learning, a FastAPI backend, database management, authentication, React frontend, live prediction, and analytics into a single platform.

10.2 Conclusion

The system successfully demonstrates how **Artificial Intelligence and Machine Learning** can be applied to healthcare data for early identification of patients who may have a higher risk of readmission.

The project integrates data from multiple hospital-related sources, performs preprocessing and feature engineering, and uses a **Logistic Regression** model to generate a readmission probability. Instead of providing only a binary prediction, the system converts the probability into **LOW, MEDIUM, and HIGH risk levels**, making the result easier to understand.

The system also provides **live prediction functionality**, allowing patient information to be entered and evaluated dynamically. The prediction results can be stored in the database and accessed through prediction history and analytics.

10.3 Key Achievements

The major achievements of the project include:

- Integration of multiple healthcare datasets.
- Data preprocessing and feature engineering.
- Time-based training and testing methodology.
- Comparison of multiple machine learning algorithms.
- Selection of Logistic Regression as the primary model.
- Probability-based readmission risk prediction.
- LOW, MEDIUM, and HIGH risk classification.
- Live patient prediction functionality.
- FastAPI-based REST API implementation.
- Database integration for patient and prediction records.
- Secure user authentication and role-based access.

- React-based user interface.
- Prediction history and analytics dashboard.

10.4 Final Outcome

The developed system provides a practical foundation for **AI-assisted hospital readmission risk assessment**. It can help healthcare organizations identify potentially high-risk patients and support better monitoring and follow-up planning.

The system is intended as a **decision-support tool rather than a replacement for healthcare professionals**. With future improvements such as real-time EHR integration, Explainable AI, advanced machine learning models, cloud deployment, automated alerts, and continuous model monitoring, the system can be further developed into a scalable healthcare intelligence platform.

REFERENCES

1. World Health Organization (WHO), *Patient Safety and Quality of Care*, World Health Organization.
2. World Health Organization (WHO), *Global Patient Safety Action Plan 2021–2030*, World Health Organization.
3. Centers for Medicare & Medicaid Services (CMS), *Hospital Readmissions Reduction Program*, U.S. Department of Health and Human Services.
4. scikit-learn Documentation, *Logistic Regression and Machine Learning Model Evaluation*, scikit-learn.
5. XGBoost Documentation, *XGBoost: Scalable and Flexible Gradient Boosting*, XGBoost Developers.
6. SHAP Documentation, *SHAP: Explainable AI and Model Interpretability*, SHAP Developers.
7. FastAPI Documentation, *FastAPI – Modern Python Web Framework*, FastAPI.
8. React Documentation, *React – A JavaScript Library for Building User Interfaces*, React Developers.
9. PostgreSQL Documentation, *PostgreSQL Database Management System*, PostgreSQL Global Development Group.
10. SQLAlchemy Documentation, *SQLAlchemy Python SQL Toolkit and Object Relational Mapper*, SQLAlchemy Developers.
11. Python Software Foundation, *Python Documentation*, Python Software Foundation.
12. W3C, *HTML and CSS Web Standards*, World Wide Web Consortium.
13. Géron, A., *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, O'Reilly Media.
14. Han, J., Kamber, M., and Pei, J., *Data Mining: Concepts and Techniques*, Morgan Kaufmann.
15. Hastie, T., Tibshirani, R., and Friedman, J., *The Elements of Statistical Learning*, Springer.
16. James, G., Witten, D., Hastie, T., and Tibshirani, R., *An Introduction to Statistical Learning*, Springer.
17. Pedregosa, F. et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*.

